

Cours de cryptographie

Balthazar Bauer

9 décembre 2025

Chapitre 1

Le formalisme en cryptographie (5 heures)

On a tous une intuition sur ce qu'est la chaleur, la taille ou la masse, mais définir formellement ces grandeurs n'a rien d'un processus trivial, et a été le fruit de nombreuses réflexions et recherches.

De manière analogue la notion de sécurité entre en résonance avec une définition intuitive.

Le flou de cette définition ne convient pas à la sensibilité du domaine. Si des développeurs ne sont pas d'accord sur ce que signifie "coder proprement", ce n'est pas forcément grave pour les utilisateurs. En revanche il est très important lors de l'usage d'un produit comportant des dangers qu'on soit bien au clair sur quels usages sont dangereux, et lesquels ne le sont pas.

Au flou va se coupler également la polysémie de la définition de sécurité. Il est évident que la sécurité d'une centrale nucléaire (même en ignorant ce qu'on entend par là), n'est pas la même que la sécurité d'une messagerie numérique.

Dans ce chapitre nous allons nous focaliser sur un contexte précis qui correspond à un usage facile à comprendre : les signatures numériques.

Mais avant de parler de sécurité, nous allons devoir préciser de quoi on parle.

1.1 Physiologie d'un schéma de signature numérique

En général un protocole de signatures consiste en deux phases : une procédure de signature, et une procédure de vérification de validation de la signature (on dira par abus de langage qu'on vérifie la signature). Étant donné qu'on est dans un cadre numérique, ces procédures consistent à appliquer un algorithme (ou une machine de Turing si l'on veut se référer au cadre usuel de l'informatique théorique) qui prennent en entrée et renvoie une donnée numérique.

Définition 1. *Un schéma de signature numérique sur l'ensemble des messages $\mathcal{M}$ est un triplet d'algorithmes probabilistes "efficaces" :*

- **KeyGen()** qui ne prend pas d'entrée et qui renvoie une paire de clefs (sk, vk) , la première clef est une clef de signatures (appelée génériquement clef secrète ou clef privée), tandis que vk est une clef de vérification (appelée aussi génériquement clef publique).
- **Sign** (sk, m) qui prend en entrée une clef de signatures sk un message $m \in \mathcal{M}$, et renvoie une signature σ .
- **Verify** (vk, m, σ) qui prend en entrée une clef de vérification, un message m , une signature σ et renvoie un bit $b \in \{0, 1\}$. Le bit ici doit être compris comme un bit logique qui nous indique si la signature est valide. C'est à dire qu'il renvoie 1 si c'est valide et 0 sinon.

Pour bien comprendre informellement de quoi on parle faisons un parallèle avec les signatures manuscrites qu'on utilise encore de nos jours assez couramment.

Le concept de signature manuscrite repose sur une capacité à reproduire un certain motif. Cette capacité est censée caractériser notre identité. Pour une signature numérique, ce qui va caractériser une identité est la connaissance d'un certain secret c'est à dire de la clef de signature sk .

Remarque Évidemment, il s'agit ici d'une identité numérique et non d'une identité physique.

Remarque On laisse pour le moment flou la notion "d'efficacité", mais on peut imaginer qu'elle correspond à une norme industrielle de type "s'exécute en moins d'un dixième de seconde sur n'importe quel ordinateur de moins de 20 ans."

Pour comprendre le cas d'usage on est ici dans un cadre *asymétrique*. C'est à dire qu'on prend en compte deux types d'entités un signataire et un vérifieur, et ces derniers n'ont *pas accès aux mêmes informations* :

Le signataire utilise sa clef secrète pour signer un message en particulier et les vérifieurs utilisent la clef de vérification (qui en pratique est associée à l'utilisateur-signataire) pour vérifier que le message a été bien signé.

Afin que le schéma puisse être utilisé ainsi, il est nécessaire qu'il vérifie une propriété qu'on appelle *propriété de correction*.

Définition 2. On dit qu'un schéma de signature numérique Π_S est correct si et seulement si : Pour toute paire (sk, vk) générée par **KeyGen**, pour tout message $m \in \mathcal{M}$:

$$\Pi_S.\text{Verify}(vk, m, \Pi_S.\text{Sign}(sk, m)) = 1.$$

1.2 Définissons la sécurité pas à pas

Il y a trois écueils à éviter lorsqu'on construit un modèle de sécurité :

- Déjà, le manque de formalisme qui rend le modèle imprécis.
- Si intrinsèquement le modèle de sécurité contredit les propriétés de correction, c'est à dire rendent inopérant tout schéma qui le satisfait. Par exemple si je considère qu'une centrale nucléaire est sécurisée uniquement si il n'y a aucun humain dans les 500 kilomètres à la ronde du réacteur, alors qu'il est nécessaire pour la faire fonctionner (et donc qu'elle soit utile) que des agents travaillent à proximité, on peut dire que le modèle est *structurellement trop fort*.
- Si on se rend compte qu'en pratique le modèle ne couvre pas des stratégies d'attaques tout à fait faisables en pratique. On va considérer que le modèle est *trop faible*.

Remarque On notera qu'un modèle *structurellement trop fort* n'a aucun intérêt dans l'absolu (car l'ensemble des constructions correctes qui le vérifie est vide). En revanche un modèle trop faible peut avoir un intérêt si on change de contexte.

Puisqu'il faut toujours partir de quelque part, on va commencer avec cette définition :

Définition 3. Un schéma Π_S est sécurisé si aucun individu malveillant ne peut signer un message qu'il n'est pas censé signer.

On peut remarquer que dans ce cours, on appelle sécurité le fait de prévenir contre des risques d'actes malveillants prémédités, et non contre des aléas "naturels" (comme la météo, le bruit électrique), où là on parlerait plus de sûreté (*safety* en anglais). On va commencer par évacuer toutes notions morales qui seraient beaucoup trop compliquées à définir. D'une manière général dans le domaine de sécurité on ne présuppose jamais des intentions finales de l'adversaire.

Définition 4. Un schéma Π_S est sécurisé si aucun individu ne peut signer un message qu'il n'est pas censé signer.

On remarque qu'on a alors renforcé la définition paradoxalement, puisque la classe d'individus ne pouvant pas signer un message augmente. Mais qu'on s'interdit d'avoir une propriété magique de type *si un individu est bien intentionné il peut alors signer un message*.

C'est peut-être malheureux, mais c'est quelque chose qu'on admet. Tout le monde accepte l'idée que si il fait claquer une porte blindée en laissant la clef à l'intérieur, il ne peut plus rentrer sans endommager la porte.

A présent, réfléchissons à ce que signifie "réussir à signer un message". Il ne s'agit pas forcément de dire que l'individu utilise l'algorithme **Sign** puisqu'on ne regarde pas précisément l'exécution qui a généré la signature (de la même manière qu'on ne vous demande pas une vidéo en train de signer manuscritement un document attaché au document).

Évidemment, on doit considérer dans notre définition l'ensemble de messages $\mathcal{M}$ duquel dépend le schéma de signature.

On va reprendre notre analogie, avec les signatures manuscrites, une signature est valide si la personne qui la vérifie est convaincue ; Étant donné qu'on est dans un cadre numérique, la vérification consiste à appliquer une procédure algorithmique prenant en entrée le message et la signature. Dans notre cas cela va donc donner :

Définition 5. Un schéma Π_S est sécurisé si pour tout message $m \in \mathcal{M}$, aucun individu ne peut générer une signature σ , telle que $\Pi_S.\text{Verify}(vk, m, \sigma) = 1$.

Maintenant il serait bon de définir comment on sait que quelqu'un est censé signer un message ou pas.

Étant donné la propriété de correction qu'on a définie, on sous-entend qu'au moins tout utilisateur qui a la clef secrète sk peut signer. On va partir sur cet ensemble et donc les utilisateurs qui ne sont pas censés signer sont ceux qui n'ont pas la clef secrète.

En revanche l'adversaire (l'individu qui cherche à *casser le schéma*) est censé avoir accès à toutes les données publiques, en particulier à vk .

Comme dans les signatures “de la vie de tous les jours”, ceux qui sont susceptibles de vérifier un document ne sont pas nécessairement les individus qui sont officiellement en capacité de signer, c'est à dire que nous sommes dans un cadre *asymétrique*¹.

Définition 6. Un schéma Π_S est sécurisé si pour tout message $m \in \mathcal{M}$ aucun individu possédant une clef vk ne peut générer une signature σ , telle que $\Pi_S(vk, m, \sigma) = 1$.

Bien sur il ne faut pas oublier de préciser l'origine de la clef c'est à dire qu'il est le résultat d'une exécution de l'algorithme **KeyGen**.

Définition 7. Un schéma Π_S est sécurisé si pour tout message $m \in \mathcal{M}$, pour toute clef vk générée par $\Pi_S.\text{KeyGen}$, aucun individu possédant vk ne peut générer une signature σ , telle que $\Pi_S.\text{Verify}(vk, m, \sigma) = 1$.

Maintenant, il serait bon de se passer la notion d'individu qui n'a pas de réalité mathématique. On va donc utiliser la notion d'algorithme en nous basant sur l'idée que ça représente bien les capacités calculatoires d'un humain du vingt-et-unième siècle, c'est à dire une machine qui prend en entrée des données (ici vk), fait une série de calculs élémentaires, et renvoie une autre donnée.

Définition 8. Un schéma Π_S est sécurisé si pour tout message $m \in \mathcal{M}$, pour toute clef vk générée par $\Pi_S.\text{KeyGen}$, aucun algorithme possédant vk ne peut générer une signature σ , telle que $\Pi_S.\text{Verify}(vk, m, \sigma) = 1$.

Remarque Ici on ne précise pas, mais par algorithme on entend selon votre contexte préféré : Machines de Turing probabilistes, fonctions récursives, fonctions de lambda calcul, machines RAM ou tout paradigme de calcul universel ayant la particularité d'avoir été correctement et rigoureusement définis formellement, et censé être “équivalent” aux capacités de calcul d'un homo sapiens (et donc équivalents à tous les modèles cités).

Malheureusement, cette définition semble un peu trop puissante, en effet on peut toujours programmer un algorithme trivial comme l'attaque $\mathcal{FB}$ de la figure 1.1 qui fait une recherche exhaustive parmi les chaînes de caractères σ , d'une qui vérifie $\Pi_S.\text{Verify}(vk, m, \sigma)$.

Heureusement en pratique un adversaire n'a pas une capacité de temps infinie.

Ici on ne va pas chercher à définir précisément les capacités de l'adversaire, puisque ça va dépendre du contexte. En effet, un pirate informatique avec un ordinateur portable n'a pas la même puissance de calcul que la NSA.

De plus certaines signatures n'ont qu'une valeur temporelle assez limitée. Par exemple sur internet dans certains protocoles, il y a des clefs éphémères valides seulement le temps d'une session de connexion.

Ici on va rester assez abstrait, et on va paramétrer la sécurité par rapport à une puissance de calcul donnée et on laissera aux utilisateurs le soin de choisir les paramètres qui les intéressent selon l'application qui leur convient.

A noter qu'on pourrait affiner la définition en prenant en compte d'autres types de ressources que le temps de calcul (la mémoire par exemple).

1. Pour voir l'équivalent *symétrique*, lire le chapitre 3 sur les codes d'authentification de messages.

Remarque En général pour estimer des grandeurs universelles, on utilise des bornes physiques par rapport à l'univers : son âge (en secondes) et son nombre d'atomes. Par exemple, si on est dans un scénario du pire-cas où tous les atomes sont utilisés comme des unités de calculs (c'est à dire comme des processeurs), et peuvent faire autant d'opérations que l'âge de l'univers en secondes, on arrive à borner cette capacité de calcul par 2^{128} opérations élémentaires, mais des estimations un peu plus fines considèrent également les grandeurs 2^{64} ou 2^{80} .

Définition 9. Soit $t \in \mathbb{N}$. Un schéma Π_S est t -sécurisé si pour tout message $m \in \mathcal{M}$, pour toute clef vk générée par $\Pi_S.\text{KeyGen}$, aucun algorithme utilisant un temps au plus t possédant vk ne peut générer une signature σ , telle que $\Pi_S.\text{Verify}(vk, m, \sigma) = 1$.

On avance, mais ce n'est toujours pas suffisant. En effet un adversaire particulièrement chanceux peut tirer au hasard la signature valide en très peu de temps.

Évidemment, sa probabilité de succès sera très faible, mais ce cas n'est pas encore pris en compte par l'actuelle définition.

Une fois de plus, on peut définir un seuil de probabilité à partir duquel on tolère que l'adversaire puisse casser un système.

Et comme pour le temps de calcul, on va éluder le problème de déterminer ce seuil en le posant en paramètre.

Définition 10. Soit t un entier naturel, $\epsilon \in [0; 1]$ Un schéma Π_S est (t, ϵ) -sécurisé si pour tout message $m \in \mathcal{M}$, pour toute clef vk générée par $\Pi.\text{KeyGen}$, pour tout algorithme probabiliste $\mathcal{A}$, qui s'exécute en temps au plus t :

$$\Pr [\Pi_S.\text{Verify}(vk, m, \sigma) = 1] \leq \epsilon.$$

A noter que la probabilité se fait par rapport à l'aléa interne de l'adversaire.

Malheureusement, une fois de plus il existe toujours un attaquant structurel, celui qui a accès à sk codé en dur, par exemple $\mathcal{A}(vk) : \Pi.\text{Sign}(sk, m)$.

On peut naïvement chercher à indiquer que $\mathcal{A}$ peut être n'importe quel algorithme sauf celui là, mais ça ne règle rien, on peut artificiellement créer un algorithme qui est complètement équivalent sans être précisément celui là (par exemple : $\mathcal{A}(vk) : \Pi.\text{Sign}(sk, m) \oplus (1 - 1 + 0 * 2^{54}).$)

Ainsi encore une fois aucun schéma ne peut vérifier cette propriété de sécurité. Elle est donc trop forte.

La vérité étant qu'a priori $\mathcal{A}$ est conçu indépendamment de (sk, vk) . Et donc tout se passe comme si KeyGen était exécuté après qu' $\mathcal{A}$ ait été défini.

Remarque A noter qu'on ne peut pas faire le même raisonnement par rapport à Π_S , qui contrairement à la clef secrète sk est censée être publiquement accessible en amont (en cohérence avec le principe de Kerckhoffs).

Définition 11. Soit t un entier naturel, $\epsilon \in [0; 1]$ Un schéma Π_S est (t, ϵ) -sécurisé si pour tout message $m \in \mathcal{M}$, pour tout algorithme probabiliste $\mathcal{A}$, qui s'exécute en temps au plus t :

$$\Pr \left[(sk, vk) \leftarrow \Pi_S.\text{KeyGen}() \right. \\ \left. \Pi_S.\text{Verify}(vk, m, \sigma) = 1 \right] \leq \epsilon.$$

A ce moment là, on va chercher à être un peu plus compact et lisible en écrivant cette propriété de manière algorithmique avec le jeu Inforg01 de la figure 1.1 :

D'une manière générale le jeu (parfois appelé expérience) est en fait un algorithme qui prend en entrée le schéma ainsi que l'adversaire et simule un scénario d'attaque censé modéliser une attaque du monde réel.

Et donc la nouvelle définition sera :

Définition 12. Soit t un entier naturel, $\epsilon \in [0; 1]$. Un schéma Π_S est (ϵ, t) -sécurisé si pour tout algorithme probabiliste $\mathcal{A}$, qui s'exécute en temps au plus t et pour tout message $m \in \mathcal{M}$:

$$\Pr [\text{Inforg01}_{\mathcal{A}, \Pi, m}() = 1] \leq \epsilon.$$

Imaginons qu'on a un schéma Π_S qui est sécurisé par rapport à $\mathcal{M}$, l'ensemble des chaînes de caractères, selon la définition 10. On va construire à partir de lui un schéma un peu particulier Π'_S identique à Π_S à l'exception de l'algorithme de vérification :

$\mathcal{FB}_{\Pi,m}(vk) :$ $\sigma := \epsilon$ Tant que $\Pi.\text{Verify}(vk, m, \sigma) \neq 1$ $\sigma \xleftarrow{\$} \cup_{i=0}^t \{0, 1\}^i$ ^a Retourner σ a. ici t représente la taille maximale d'une signature	$\text{Inforg01}_{\mathcal{A},\Pi,m}() :$ $(sk, vk) \leftarrow \Pi.\text{KeyGen}()$ $\sigma \leftarrow \mathcal{A}(vk)$ Retourner $\Pi.\text{Verify}(vk, m, \sigma)$	$\text{Inforg02}_{\mathcal{A},\Pi}() :$ $(sk, vk) \leftarrow \Pi.\text{KeyGen}()$ $(m, \sigma) \leftarrow \mathcal{A}(vk)$ Retourner $\Pi.\text{Verify}(vk, m, \sigma)$
---	---	--

FIGURE 1.1 – Attaquant *ForceBrute*, et Jeux d'inforgeabilité : Versions 0.1 et 0.2

— $\Pi'_S.\text{Verify}(vk, m, \sigma) :$ Si $m = \text{"Moi l'utilisateur ayant pour clef } vk \text{ j'ai volé un yaourt à la cantine."}$, on renvoie 1, sinon on renvoie $\Pi_S(vk, m, \sigma)$.

Si on fixe le message à l'avance

$m = \text{"Moi l'utilisateur ayant pour clef A07618E918CD180DDB19911, j'ai volé un yaourt à la cantine."}$

Il est improbable que l'adversaire réussisse à forger une signature par rapport à ce message, car le seul cas où il est pourra est lorsque la clef de vérification vk devient A07618E918CD180DDB19911 au moment de l'exécution de KeyGen , ce qui arrive avec une faible probabilité.

Donc ce schéma Π'_S est sécurisé par rapport à la définition 10, or il ne l'est pas d'un point de vue concret, car on ne pourra considérer comme authentique aucun aveu de vol de yaourt.

On se rend compte que m ne peut être déterminé en amont. Car on risque de ne pas prendre en considération les messages dépendants des clefs.

On va donc changer le jeu d'inforgeabilité en passant de la version 0.1 à la version 0.2 (cf figure 1.1), et ne plus mentionner de messages dans la définition :

Définition 13. Soit t un entier naturel, $\epsilon \in [0; 1]$. Un schéma Π_S est (ϵ, t) -sécurisé si pour tout algorithme probabiliste $\mathcal{A}$, qui s'exécute en temps au plus t : $\Pr [\text{Inforg02}_{\mathcal{A},\Pi}() = 1] \leq \epsilon$.

Il est clair que si on veut que les signatures soient comparables avec les signatures manuscrites, on va avoir a priori signer plusieurs documents et certaines signatures seront potentiellement accessibles à autrui.

Il est donc nécessaire si on veut éviter le troisième écueil, c'est à dire que notre modèle ne soit pas trop faible et reste pertinent qu'on donne un accès à des signatures à l'adversaire.

La première tentation est de chercher à coller le plus possible à la réalité en essayant d'imaginer quelles personnes auront accès aux signatures de quels messages : Évidemment cela va dépendre du contexte : Par exemple si on est dans le cadre de signatures de courriels d'un francophone très poli et soucieux de la grammaire nommé Didier, on va essayer d'imaginer tous les types de messages ayant le format d'un courriel écrit en français qui commence par $S_{\text{debut}} := \{\text{Chère Madame, Cher Monsieur}\}$, termine par un élément de $S_{\text{fin}} := \{\text{Cordialement Didier, Respectueusement Didier}\}$. et contient entre ses deux séparateurs : S_3 l'ensemble des textes grammaticalement correct.

On va donc appeler cet ensemble de messages plausibles $\mathcal{M}_{\text{courriels de Didier}}$.

Évidemment, il ne s'agit pas de dire que l'adversaire a accès à toutes les signatures d'éléments de $\mathcal{M}_{\text{courriels de Didier}}$, car ce nombre étant infini, cela contredirait la borne temporelle de son temps de calcul.

On peut d'abord se dire qu'on va chercher à lui donner accès à des messages aléatoires, mais on cela complexifie encore le modèle puisqu'il faut définir une distribution de probabilité.

En fait, à ce stade là le modèle est trop compliqué : Une solution radicale apparaît : laisser à l'adversaire le choix des messages qu'il veut signer. Cela couvre largement le cas d'usage énoncé mais également bien d'autres : Par exemple en envoyant certains messages à Didier qui vont provoquer avec forte probabilité certaines réponses, il est facile d'imaginer que l'adversaire a une prise sur les messages pour lesquels il pourra accéder à leurs signatures.

Par ailleurs toujours dans ce paradigme "simplification-renforcement," on va écarter l'idée d'un ensemble de messages $\mathcal{M}_{\text{courriels de Didier}}$ qui dépendent du contexte pour se baser uniquement sur l'ensemble $\mathcal{M}$ des messages concernés par la propriété de correction du schéma Π_S . On simplifie tout en étant bien moins dépendant du contexte.

En nous inspirant de la théorie de la complexité, nous allons utiliser un oracle, c'est à dire une petite machine extérieure à l'adversaire et avec laquelle il peut interagir.

<u>Inforg03_{A,Π}() :</u>	<u>A₁^O(vk) :</u>
$(sk, vk) \leftarrow \Pi.\text{KeyGen}()$	$m \xleftarrow{\$} \mathcal{M}$
$(m, \sigma) \leftarrow \mathcal{A}^{\text{Sign}(sk, \cdot)}(vk)$	$\sigma \xleftarrow{\$} O(m)$
Retourner $\Pi.\text{Verify}(vk, m, \sigma)$	Retourner (m, σ)

FIGURE 1.2 – Version 0.3 du jeu d’inforgeabilité, et adversaire $\mathcal{A}_1$ structurel de ce jeu

<u>Inforg04_{A,Π}() :</u>	<u>Inforg_{A,Π}(λ) :</u>	<u>OSign_{sk}(m) :</u>
$(sk, vk) \leftarrow \Pi.\text{KeyGen}()$	$(sk, vk) \leftarrow \Pi.\text{KeyGen}(1^\lambda)$	$Q := Q \cup \{m\}$
$Q := \emptyset$	$Q := \emptyset$	Retourner
$(m, \sigma) \leftarrow \mathcal{A}^{OSign_{sk}(\cdot)}(vk)$	$(m, \sigma) \leftarrow \mathcal{A}^{OSign_{sk}(\cdot)}(vk)$	$\Pi.\text{Sign}(sk, m)$
Retourner	Retourner	
$\Pi.\text{Verify}(vk, m, \sigma) \wedge (m \notin Q)$	$\Pi.\text{Verify}(vk, m, \sigma) \wedge (m \notin Q)$	

FIGURE 1.3 – Versions 0.4, et 1.0 du jeu d’inforgeabilité, avec l’oracle de signatures

En l’occurrence l’oracle de la figure 1.1 utilise la clef de signature (auquel l’adversaire n’a pas accès) pour signer des messages.

Malheureusement, on va alors tomber dans le deuxième écueil, c’est à dire qu’aucun schéma ne peut vérifier cette propriété, car si le schéma est correct (c’est à dire fonctionnel) alors l’adversaire structurel $\mathcal{A}_1$ de la figure 1.2 pourra toujours forger une signature valide.

Doit-on renoncer à la sécurité des signatures numériques ou à leur formalisation ? En fait non, car si on se penche sur ce que fait $\mathcal{A}_1$, on se rend compte que son “attaque” ne devrait pas être considérée comme telle. En effet, il n’y a rien de choquant à ce qu’on puisse retrouver un document signé, si il a effectivement été signé par le passé. On peut argumenter que peut-être ce document n’est plus valide. Mais alors dans la vraie vie, on gère ce genre de chose autrement qu’avec juste une signature. En effet tant qu’on parle de données numériques classiques, il est toujours possible de les copier et de les renvoyer à un autre moment. On doit donc gérer ce genre de chose en datant les messages par exemple, ou en précisant qu’une clef de vérification vk a une “date de péremption”.

Si on veut réparer notre modèle, il va donc falloir considérer l’ensemble Q des messages qui ont été signés, puis vérifier que les signatures sont considérées comme des attaques uniquement si elles concernent des messages hors de cet ensemble.

On va donc utiliser le jeu Inforg04 de la figure 1.3 :

Définition 14. Soit t un entier naturel, $\epsilon \in [0; 1]$. Un schéma Π_S est (ϵ, t) -sécurisé si pour tout algorithme probabiliste $\mathcal{A}$, qui s’exécute en temps au plus t : $\Pr [\text{Inforg04}_{\mathcal{A}, \Pi}() = 1] \leq \epsilon$.

A présent que nous avons une définition qui semble satisfaire les trois conditions qu’on s’étaient fixées, le lecteur pourra se familiariser avec l’exercice suivant :

Exercice 1. Soit $(\epsilon, \epsilon') \in [0; 1]^2$, $(t, t') \in \mathbb{N}^2$. Soit Π_S , et Π'_S deux systèmes de signatures numériques respectivement (ϵ, t) -sécurisé, et (ϵ', t') -sécurisé par rapport à la définition 14.

On pose Π_{concatet} :

KeyGen : $(sk, vk) \leftarrow \Pi_S.\text{KeyGen}()$, $(sk', vk') \leftarrow \Pi'_S.\text{KeyGen}()$. Retourner $((sk, sk'), (vk, vk'))$.

Sign $((sk, sk'), m)$: $\sigma \leftarrow \Pi_S.\text{Sign}(sk, m)$; $\sigma' \leftarrow \Pi'_S.\text{Sign}(sk', m)$. Renvoyer (σ, σ') .

Verify $((vk, vk'), m, (\sigma, \sigma'))$: Retourner $\Pi_S.\text{Verify}(vk, m, \sigma) \wedge \Pi'_S.\text{Verify}(vk', m, \sigma')$.

Montrer qu’il existe une constante K tel que Π_{concatet} est $(\max(\epsilon, \epsilon'), \min(t, t') - K)$ -sécurisé.

1.3 De la sécurité concrète à la sécurité asymptotique : Universalisons la définition

Si lorsqu’on utilise un produit cryptographique directement pour une application concrète la sécurité concrète est sans doute la plus pertinente. Mais lors d’une première analyse théorique, on souhaiterait avoir quelque chose de plus universel, c’est à dire de pouvoir se débarrasser au maximum des paramètres ϵ et t .

On part donc de ce genre de définition :

Définition 15 (Sécurité Concrète). *Un schéma est dit (t, ϵ) -sécurisé si pour tout algorithme ayant un temps d'exécution t , la probabilité que ce dernier "casse" le schéma est bornée par ϵ .*

En faisant l'exercice 1, on se rend compte que l'évaluation de la sécurité de combinaisons de plusieurs schéma sécurisés peut s'avérer assez complexe à définir notamment à cause de l'éternel problème de la non universalité d'une métrique temporelle. Pour remédier à ça, on va s'inspirer de ce qui se fait usuellement en théorie de la complexité : Si l'évaluation du temps d'un algorithme pseudo-code est très dépendant du modèle de calcul considéré, de la manière dont on définit le temps de calcul ou du langage de programmation dans lequel il sera codé. En revanche si on fait varier la taille de l'entrée et qu'on regarde asymptotiquement la courbe du temps d'exécution de l'algorithme, alors quel que soit le langage ou modèle ou définition de temps choisi, le temps de calcul sera le même à une constante multiplicative près.

Dans ce paradigme, tous les algorithmes linéaires (c'est à dire ayant un temps de calcul linéaire en la taille de l'entrée) sont considérés comme efficaces, car on sait qu'ils n'auront pas de problème à être exécutés par une machine capable de lire l'entrée : Évidemment si la constante multiplicative est énorme, ce ne sera pas le cas, mais aucun algorithme conçu *naturellement* (c'est à dire pas un exemple pathologique) n'a eu une constante qui rend un calcul inaccessible.

Les algorithmes linéaires vérifient la bonne propriété qu'ils se combinent relativement bien : si une procédure consiste en la concaténation de trois fois l'exécution d'un algorithme linéaire puis une fois un autre algorithme linéaire, alors la procédure sera elle même linéaire.

Malheureusement, les théoriciens de la complexité se sont vite rendus compte que certaines combinaisons ne fonctionnaient pas. Par exemple si je fais un nombre d'appel linéairement grand à une fonction qui n'a pas un coût constant, mais un coût linéaire ou même logarithmique, la procédure ne sera plus un algorithme linéaire.

Ainsi, il a été décidé de considérer les calculs de temps polynomial en la taille de l'entrée comme efficace. Si en faisant ça on se démarque beaucoup plus de la pratique qu'en négligeant seulement les constantes multiplicatives, il se trouve qu'en *première analyse*, cela reste pertinent².

Ainsi, on peut donc définir ce qu'est un attaquant efficace et évacuer toute notion floue concernant le temps de la définition de sécurité, en disant qu'un attaquant efficace est un algorithme polynomial. A noter qu'ici le fait de prendre en compte *trop* d'algorithmes, c'est à dire des algorithmes avec des trop grosses constantes ou de trop gros exposants n'est pas du tout un problème, cela ne fait que renforcer notre définition de sécurité³.

Définition 16. Soit $\epsilon \in [0; 1]$. Un schéma Π_S est ϵ -sécurisé si pour tout algorithme probabiliste borné polynomialement $\mathcal{A}$: $\Pr [\text{Inforg04}_{\mathcal{A}, \Pi}() = 1] \leq \epsilon$.

Mais cette définition n'est pas consistante, en effet, à partir du moment où le schéma est fixé, on a donc une borne sur la taille clef publique, les tailles de signatures (par rapport au fait qu'on impose que l'exécution de $\Pi.\text{Sign}$ soit rapide), et un temps d'exécution *constant* de $\Pi.\text{Verify}(vk, m, \cdot)$, donc l'attaquant $\mathcal{FB}$ de la figure 1.1 pour n'importe quel message fixé $m \in \mathcal{M}$ est en fait un attaquant en temps constant (qui n'est même pas défini pour des entrées à partir d'une certaine taille), et donc va casser à nouveau *universellement* la propriété d'inforgeabilité.

A ce moment là, il est très tentant de revenir sur notre définition originelle de schéma de signature.

Et nous allons donc céder à la tentation en prenant le risque de se détacher des schémas de signature du monde réel (mais sans perdre des yeux ce risque).

En fait on va essayer de se sortir par le haut de ce problème ("tout ce qui ne tue pas..." tout ça, tout ça). Le problème étant que tant que les signatures auront une taille bornée l'attaquant $\mathcal{FB}$ mettra en danger notre définition. Il est donc nécessaire que la taille de la sortie de $\Pi.\text{Sign}$ grandissent. On va donc laisser tomber la définition pas assez formelle "s'exécute en moins d'une seconde", et dire que $\Pi.\text{Sign}$ est aussi un algorithme PPT (c'est à dire un algorithme probabiliste qui s'exécute en un temps polynomial).

2. La grande majorité des problèmes du monde réel ayant une résolution polynomiale se sont trouvés avoir des algorithmes ayant des temps de calcul bornables par des polynômes de petits degrés (2 ou 3)

3. Enfin pour être plus précis cela pourrait être un problème si aucun schéma ne vérifierait la propriété à cause de l'existence d'un attaquant universel peu réaliste (car avec un exposant très gros par exemple), mais divulgachons : ça n'arrivera pas.

Mais ça ne suffit pas car son entrée est de taille bornée (si on fixe un message). En effet vk est générée par $\Pi.\text{KeyGen}$, qui elle même doit s'exécuter en moins d'une seconde. Il est donc également nécessaire que la taille de vk puisse grossir et donc que $\Pi.\text{KeyGen}$ soit considéré aussi comme un PPT.

Sauf que $\Pi.\text{KeyGen}$ ne prend pas d'entrée dans la définition actuelle. Nous allons donc lui donner artificiellement une entrée dont la taille est variable. Comme seule la taille de l'entrée est pertinente pour cet algorithme on décide de lui donner en entrée uniquement des 1 (par convention).

Ainsi les tailles de clefs et de signatures vont donc grossir polynomialement⁴ avec le nombre de 1 de l'entrée de $\Pi.\text{KeyGen}$.

Du coup, comme la taille de ses entrées (aussi bien la clef de vérification que la signature) ne sont plus bornées par une constante, il n'est plus possible que le temps d'exécution de $\Pi.\text{Sign}$ soit borné par une constante, et on va donc logiquement se contenter d'exiger qu'il soit polynomialement borné en la taille de l'entrée (et donc soit un PPT), ce qui achève la formalisation de ce qu'est un schéma de signatures numériques.

Définition 17. *Un schéma de signature numérique sur l'ensemble de messages $\mathcal{M}$ est un triplet d'algorithmes PPT :*

- $\text{KeyGen}(1^\lambda)$ qui prend en entrée un paramètre de sécurité écrit en unaire et qui renvoie une paire de clefs (sk, vk) , la première clef est une clef de signatures, tandis que vk est une clef de vérification.
- $\text{Sign}(sk, m)$ qui prend en entrée une clef de signatures sk un message $m \in \mathcal{M}$, et renvoie une signature σ .
- $\text{Verify}(vk, m, \sigma)$ qui prend en entrée une clef de vérification, un message m , une signature σ et renvoie un bit $b \in \{0, 1\}$.

On doit donc légèrement modifier le jeu de sécurité pour introduire la nouvelle entrée de KeyGen , mais à présent qu'on a dit que notre adversaire était polynomialement borné, et pas seulement borné par une constante t , il sera toujours possible de créer des adversaires exhaustifs pour certains lambdas précis. Leur caractérisation de "polynomialement borné" n'ayant qu'une signification asymptotique. On va donc regarder leur probabilité de succès asymptotiquement.

Définition 18. *Soit $\epsilon \in [0; 1]$. Un schéma $\Pi_{\mathcal{S}}$ est ϵ -sécurisé si pour tout algorithme probabiliste borné polynomialement $\mathcal{A} : \lim_{\lambda \rightarrow \infty} (\Pr [\text{Inforg}_{\mathcal{A}, \Pi}(\lambda) = 1]) \leq \epsilon$.*

L'introduction de ce paramètre de sécurité peut être vu comme une abstraction de théoricien qui divague car trop éloigné du "terrain". Mais il s'avère qu'en réalité cette considération théorique correspond à une réalité concrète : Si on met plus de 1, les tailles des objets produits (ici les signatures) vont potentiellement augmenter et donc les adversaires (superpolynomiaux si la définition de sécurité est vérifiée) qui cassent le schéma vont donc voir leur temps de calcul monter.

Et si on est pas convaincu par les considération asymptotiques, mais qu'on veut quand même pouvoir dire des choses par rapport à la paramétrisation du schéma on pourra prendre cette définition :

Définition 19. *Soit $\epsilon \in [0; 1]^{\mathbb{N} \times \mathbb{N}}$. Un schéma $\Pi_{\mathcal{S}}$ est ϵ -sécurisé si pour tout algorithme probabiliste $\mathcal{A}$, qui s'exécute en temps au plus t : $\Pr [\text{Inforg}_{\mathcal{A}, \Pi}(\lambda) = 1] \leq \epsilon(t, \lambda)$ ⁵.*

A noter qu'augmenter ce paramètre n'est pas anodin, il va *a priori* augmenter le temps d'exécution des algorithmes du système. Mais comme par définition une fonction superpolynomiale (qui serait le temps de calcul d'un attaquant efficace) écrase asymptotiquement une fonction polynomiale, on sait qu'il existe un λ où les temps de calcul des algorithmes du système seront encore réalisable, alors que ceux de l'attaquant ne le seront plus⁶.

On se retrouve donc avec un paramètre qui fixe le niveau de sécurité (par rapport à la variété des contextes dont on a parlé plus tôt dans ce chapitre). On appelle donc cette grandeur le paramètre de sécurité.

4. en pratique ce sera souvent linéaire et quasiment jamais plus de quadratique

5. On peut affiner le calcul de ϵ , en prenant en compte la variété des ressources que peut utiliser $\mathcal{A}$ et pas seulement le temps de calcul. Par exemple on pourrait considérer m les ressources en mémoire qu'il utilise, ainsi que q le nombre d'appels à oracles $O\text{Sign}$, ϵ serait alors dans $[0; 1]^{\mathbb{N}^4}$.

6. En théorie, ce λ peut ne pas exister si les constantes pour les algorithmes du système sont tellement catastrophiques, que leur vitesse d'exécution est supérieur à celle d'un potentiel attaquant pour des paramètres où ils sont eux même déjà complètement inefficaces, mais généralement ça ne pose pas de problème, les courbes se croisant bien assez tôt.

$\text{InforgMU}_{\mathcal{A},\Pi}(\lambda) :$ $C \leftarrow U \leftarrow Q \leftarrow \emptyset$ $(vk, m, \sigma) \leftarrow \mathcal{A}^{\text{OCreate}(), \text{OCorrupt}(\cdot), \text{OSign}(\cdot, \cdot)}$ Retourner $\Pi.\text{Verify}(vk, m, \sigma)$ $\wedge ((vk, m) \notin Q) \wedge ((vk, \cdot) \in U)$	$\text{OCreate}() :$ $(sk, vk) \leftarrow \Pi.\text{KeyGen}(1^\lambda)$ $U := U \cup \{(sk, vk)\}$ Retourner vk $\text{OCorrupt}(vk) :$ $Q := Q \cup (\{vk\} \times \mathcal{M})$ Si $\exists sk / (sk, vk) \in U$ Retourner sk Sinon Retourner $\perp$	$\text{OSign}(vk, m) :$ $Q := Q \cup \{(vk, m)\}$ Si $\exists sk / (sk, vk) \in U$ $\sigma \leftarrow \Pi.\text{Sign}(sk, m)$ Retourner σ Sinon Retourner $\perp$
---	---	---

FIGURE 1.4 – Jeu de sécurité d’inforgeabilité dans un contexte multi-utilisateurs.

Exercice 2. Soit $(\epsilon, \epsilon') \in [0; 1]^2$. Soit Π_S , et Π'_S deux systèmes de signatures numériques respectivement ϵ -sécurisé, et ϵ' -sécurisé par rapport à la définition 18.

On pose Π_{concatou} :

$\text{KeyGen}(1^\lambda) : (sk, vk) \leftarrow \Pi_S.\text{KeyGen}(1^\lambda), (sk', vk') \leftarrow \Pi'_S.\text{KeyGen}(1^\lambda)$. Retourner $((sk, sk'), (vk, vk'))$.

$\text{Sign}((sk, sk'), m) : \sigma \leftarrow \Pi_S.\text{Sign}(sk, m); \sigma' \leftarrow \Pi'_S.\text{Sign}(sk', m)$. Retourner (σ, σ') .

$\text{Verify}((vk, vk'), m, (\sigma, \sigma')) : \text{Retourner } \Pi_S.\text{Verify}(vk, m, \sigma) \vee \Pi'_S.\text{Verify}(vk', m, \sigma')$.

Montrer que Π_{concatou} est $(\epsilon + \epsilon')$ -sécurisé. Méditer sur le fait qu’a priori, on ne pourra pas avoir mieux (en construisant des exemples pathologiques).

Si le schéma de l’exercice 2 peut paraître artificiel et n’a aucune intérêt pratique, il représente un genre de combinaisons assez courantes où l’on combine deux schémas de sécurité (souvent de types différents) afin de créer un schéma qui vérifiera de nouvelles fonctionnalités (ici ce n’est pas le cas⁷).

On a donc un problème du fait de la paramétrisation de la définition. En effet si on considère qu’il y a un seuil ϵ_{seuil} au delà duquel les propriétés. Alors, je vais avoir besoin d’établir une sécurité plus forte sur Π_S, Π'_S qu’une ϵ_{seuil} -sécurité (par exemple une $\frac{\epsilon_{\text{seuil}}}{2}$ -sécurité).

Ce qui implique a priori une nouvelle preuve de sécurité, voir un nouveau schéma. Cela peut paraître un peu frustrant de devoir reconstruire un schéma alors qu’on a déjà une définition paramétrique pour notre schéma.

On pourrait se rabattre sur la définition 19, en supposant que la fonction epsilon est la plus fine possible, on pourra le faire dans certains contextes, mais on perd en capacité d’analyse. Et surtout on s’éloigne de notre objectif d’universalité de la définition.

La solution qui apparaît la plus canonique est donc la suivante : ne considérer que les schémas 0-sécurisés :

Définition 20. Un schéma Π_S est sécurisé si pour tout algorithme probabiliste borné polynomialement $\mathcal{A}$, il existe $\epsilon \in [0; 1]^\mathbb{N}$ une suite qui tend vers 0 tel que $\Pr [\text{Inforg}_{\mathcal{A},\Pi}(\lambda) = 1] \leq \epsilon(\lambda)$.

Avec cette nouvelle définition on a enfin une notion “universelle” de la sécurité. C’est à dire qu’on a enfin déparamétré la notion de sécurité.

A présent essayons de réfléchir à un cas d’usage multi-utilisateurs de notre schéma de sécurité :

On peut regarder deux versions de définitions de sécurité :

Définition 21. Soit $\epsilon \in [0; 1]^\mathbb{N}^3$. Un schéma Π_S est ϵ -sécurisé dans le contexte multi-utilisateur si pour tout algorithme probabiliste $\mathcal{A}$, qui s’exécute en temps au plus t et fait au plus q requêtes OCreate :

$$\Pr [\text{InforgMU}_{\mathcal{A},\Pi}(\lambda) = 1] \leq \epsilon(t, q, \lambda).$$

Définition 22. Un schéma Π_S est sécurisé dans le contexte multi-utilisateurs si pour tout algorithme probabiliste polynomiales $\mathcal{A}$, il existe $\epsilon \in [0; 1]^\mathbb{N}$ une suite qui tend vers 0 tel que :

$$\Pr [\text{InforgMU}_{\mathcal{A},\Pi}(\lambda) = 1] \leq \epsilon(\lambda).$$

Exercice 3. Montrer que si Π_S est ϵ -sécurisé selon la définition 19, alors il est $q\epsilon$ -sécurisé dans le contexte multi-utilisateurs selon la définition 21.

⁷. Mais on peut toujours y voir une propriété “si-je-perds-une-moitié-de-clef-je-peux-quand-même-faire-des-signatures-valides”

Indice : On pourra essayer de deviner quel utilisateur parmi les q du système va se faire effectivement attaquer et imaginer que celui ci a les clefs du système originel du jeu Inforg, tandis que les $(q - 1)$ autres utilisateurs seront simulés artificiellement.

On se rend alors compte que l'on peut avoir un schéma sécurisé selon la définition 16. En effet le nombre de requêtes à $O\text{Create}$ pouvant être polynomial (ou dit autrement, le nombre d'utilisateurs de notre système pouvant être polynomial) on a aucune garantie que $q(\lambda) \cdot \epsilon(\lambda)$ tende vers zéro.

Être stable par addition n'est donc pas suffisant pour notre ensemble de fonctions ϵ pertinentes. On veut aussi être stable par multiplication par un polynôme en λ .

On va donc introduire l'ensemble des fonctions négligeables.

Définition 23. Une fonction f de $\mathbb{N} \rightarrow \mathbb{R}$ est négligeable si pour tout polynôme P , f est asymptotiquement bornée devant $\left(\frac{1}{P(n)}\right)_{n \in \mathbb{Z}}$.⁸

Propriété 1. Si on pose P , l'ensemble des fonctions $\mathbb{Q}$ -polynomiales définies sur $\mathbb{N}$. L'ensemble des fonctions négligeables est un P -module.

Preuve laissée en exercice pour le lecteur. □

On va donc avoir comme définition asymptotique définitive :

Définition 24. Un schéma Π_S est sécurisé si pour tout algorithme probabiliste polynomialement borné $\mathcal{A} : \Pr [\text{Inforg}_{\mathcal{A}, \Pi}(\lambda) = 1]$ est une fonction négligeable en λ .

1.4 Conclusion

Pour conclure, on va retenir deux définitions : une concrète à paramètres (la définition 19) et une définition plus universelle, et plus théorique qui prend son sens asymptotiquement (la définition 24).

La première fait office de définition “finale”, et nécessite d'avoir en tête ce qu'on prend en compte comme capacité de calcul assez précisément. Tandis que la seconde est une définition qui sert de première ébauche, afin d'établir la sécurité d'un schéma en omettant le choix des paramètres, et les capacités de calcul de l'adversaire prise en compte.

A noter que si la vision générique des définitions concrètes et universelles va s'appliquer dans énormément de contextes très différents de celui des signatures numériques. Le jeu de sécurité (appelé aussi expérience), lui en revanche doit être repensé pour chaque contexte. Et il n'y aura jamais de formule mathématique pour construire de tels jeux (de la même manière qu'il n'existe pas de formule mathématique pour construire des modèles physiques). Afin de s'exercer à la création de nouveaux modèles de sécurité, le lecteur pourra réfléchir à l'exercice suivant :

Définition 25. Un système de mot de passe est un triplet d'algorithmes PPT ($\text{Init}, \text{Add}, \text{Verify}$) :

- $\text{Init}(1^\lambda)$ prend en entrée un entier écrit en unaire, et renvoie une base de données B .
- $\text{Add}(B, id, mdp)$ prend en entrée une base de données, un identifiant et un mot de passe et renvoie une base de données, ainsi qu'un message m .
- $\text{Verify}(B, id, mdp)$ prend en entrée une base de données, un identifiant et un mot de passe et renvoie un message m .

Exercice 4. Définir une ou plusieurs propriétés de correction, ainsi que des propriétés de sécurité modélisant ce qu'on attend d'un tel système dans le monde réel.

8. Par rapport à la définition usuelle de négligeable, on peut le traduire ainsi, une fonction est négligeable en λ , si elle est négligeable en $+\infty$ par rapport à $\left(\frac{1}{|P(\lambda)|}\right)_{\lambda \in \mathbb{N}}$ pour tout polynôme P .

Chapitre 2

Chiffrement à clef secrète (7 heures)

A présent que nous sommes au point sur le formalisme des jeux de sécurité, nous allons nous intéresser à la problématique historique de la cryptographie : La confidentialité.

Il va s'agir de dissimuler de l'information sur des espaces non sécurisés (c'est à dire accessibles à des entités auxquels on ne fait pas confiance a priori).

Nous allons donc nous focaliser sur la primitive vedette de la cryptographie : Le chiffrement.

On peut retenir deux contextes très génériques dans lequel le chiffrement d'une donnée s'applique : Celui des communications sur des canaux non sécurisés (métaphore du convoyeur de fonds), celui du stockage de données sur un serveur a priori accessible à des entités auxquelles on ne fait pas confiance (métaphore du casier à la piscine). Dans les deux cas en plus de l'exigence de sécurité, on va bien sûr souhaiter l'intégrité de la donnée (de la même manière qu'un incinérateur qui vérifie une bonne propriété de sécurité en tant que casier n'est pas adapté pour servir de casier).

2.1 Premières définitions

Pour repartir sur des bases saines, et en remarquant que la problématique de la confidentialité est plus complexe à définir que l'authentification, commençons par définir les algorithmes qui caractérisent un système de chiffrement. Soit $\mathcal{M}$ l'ensemble des messages en clair.

Définition 26. *Un schéma de chiffrement est un triplet d'algorithmes $PPT : (\text{KeyGen}, \text{Enc}, \text{Dec})$.*

$\text{KeyGen} : \emptyset \rightarrow \mathcal{K}$

$\text{Enc} : \mathcal{K} \times \mathcal{M} \rightarrow \mathcal{C}$

$\text{Dec} : \mathcal{K} \times \mathcal{C} \rightarrow \mathcal{M}$

où les éléments de $\mathcal{K}$ et $\mathcal{C}$ sont respectivement appelés clefs de chiffrement (qu'on peut parfois appeler tout simplement clefs selon le contexte) ou chiffres, et chiffrés ou cryptogrammes.

Comme pour les signatures numériques vues dans le chapitre 1, avant de discuter de sécurité, il nous faut parler de ce qu'on attend comme propriété minimale afin qu'il puisse être utilisé en pratique :

Définition 27. *On dit que le schéma est correct si et seulement si $\forall (k, m) \in \mathcal{K} \times \mathcal{M} : \text{Dec}_k(\text{Enc}_k(m)) = m$.*

2.1.1 La sécurité parfaite

On peut partir sur une définition informelle qui dit qu'un schéma Π est sécurisé si on ne déduit aucune information sur un texte clair en voyant un chiffré d'un texte clair.

Ce qui d'un point de vue probabiliste peut s'exprimer de cette manière là :

Définition 28 (Sécurité de Shannon). *On dit qu'un schéma Π est confidentiel au sens de Shannon si pour toute distribution d sur $\mathcal{M}$, $\forall m \in \mathcal{M}$ et $\forall c \in \mathcal{C}$,*

$$\Pr_{k \leftarrow \Pi.\text{KeyGen}; m' \xleftarrow{d} \mathcal{M}} [m = m' | \Pi.\text{Enc}_k(m') = c] = \Pr_{m' \xleftarrow{d} \mathcal{M}} [m = m'].$$

Expliquons un peu en détail cette définition : la distribution d correspond au contexte sur lequel on applique notre méthode de chiffrement. Ce contexte est supposé connu de tous (et en particulier d'un curieux qui n'est pas censé avoir accès à la donnée). Par exemple si le message m' est *publiquement connu* pour être extrait d'une correspondance épistolaire entre deux footballeurs le lendemain d'un match, il est plus probable que le message termine par le nom du rédacteur (comme c'est l'usage pour un courrier), et que les mots "ballon", "but", "tir" soient mentionnés dans le message. Ceci est une information *a priori*, considérée comme non sensible. On va donc chercher à ce qu'une personne qui n'a aucune information sur la clef de chiffrement k ne déduise aucune information sur le message en clair m' , même si il sait que le chiffré de m' par la clef k est le cryptogramme c . Cela revient à dire que la distribution de m' ne change pas même si l'on connaît c , donc que :

$$\Pr_{k \leftarrow \Pi.\text{KeyGen}; m' \xleftarrow{d} \mathcal{M}} [m = m' | \Pi.\text{Enc}_k(m') = c] = \Pr_{m' \xleftarrow{d} \mathcal{M}} [m = m'].$$

On peut simplifier cette définition grâce à la définition de la sécurité parfaite qui permet d'évacuer la notion de distribution des messages :

Définition 29 (Sécurité parfaite). *On dit qu'un schéma Π est parfaitement sécurisé si*

$$\forall m_1, m_2 \in \mathcal{M}, c \in C, \Pr_{k \leftarrow \Pi.\text{KeyGen}} [\Pi.\text{Enc}_k(m_1) = c] = \Pr_{k \leftarrow \Pi.\text{KeyGen}} [\Pi.\text{Enc}_k(m_2) = c].$$

Propriété 2. *Les définitions 28, et 29 sont équivalentes.*

On suppose qu'on a un schéma Π parfaitement sécurisé. Soit d une distribution sur $\mathcal{M}$. Soit $m, m_2, c \in \mathcal{M} \times \mathcal{M} \times C$. Sans perdre de généralité $\exists m', k \in \mathcal{M} \times \mathcal{K}$ tel que $\Pr [\Pi.\text{Enc}_k(m') = c] \neq 0$. Donc sans perdre de généralité $k \in \Pi.\text{KeyGen}()$. Donc sans perdre de généralité $\Pr_{m' \xleftarrow{d} \mathcal{M}, k \leftarrow \Pi.\text{KeyGen}()} [\Pi.\text{Enc}_k(m') = c] \neq 0$. On peut donc faire le calcul suivant :

$$\begin{aligned} & \Pr_{m' \xleftarrow{d} \mathcal{M}, k \leftarrow \Pi.\text{KeyGen}()} [m = m' | \Pi.\text{Enc}_k(m') = c] \\ &= \frac{\Pr_{m' \xleftarrow{d} \mathcal{M}, k \leftarrow \Pi.\text{KeyGen}()} [m = m' \wedge \Pi.\text{Enc}_k(m') = c]}{\Pr_{m' \xleftarrow{d} \mathcal{M}, k \leftarrow \Pi.\text{KeyGen}()} [\Pi.\text{Enc}_k(m') = c]} \\ &= \frac{\Pr_{m' \xleftarrow{d} \mathcal{M}, k \leftarrow \Pi.\text{KeyGen}()} [m = m' \wedge \Pi.\text{Enc}_k(m) = c]}{\Pr_{m' \xleftarrow{d} \mathcal{M}, k \leftarrow \Pi.\text{KeyGen}()} [\Pi.\text{Enc}_k(m') = c]} \\ & \text{Comme l'aléa de la clef et de Enc est indépendant de celui de } d : \\ &= \frac{\Pr_{m' \xleftarrow{d} \mathcal{M}} [m = m'] \Pr_{k \leftarrow \Pi.\text{KeyGen}()} [\Pi.\text{Enc}_k(m) = c]}{\Pr_{m' \xleftarrow{d} \mathcal{M}, k \leftarrow \Pi.\text{KeyGen}()} [\Pi.\text{Enc}_k(m') = c]}. \end{aligned} \tag{2.1}$$

Or

$$\begin{aligned} & \Pr_{m' \xleftarrow{d} \mathcal{M}, k \leftarrow \Pi.\text{KeyGen}()} [\Pi.\text{Enc}_k(m') = c] \\ &= \sum_{m'' \in \mathcal{M}} \Pr_{m' \xleftarrow{d} \mathcal{M}} [m'' = m'] \Pr_{k \leftarrow \Pi.\text{KeyGen}()} [\Pi.\text{Enc}_k(m'') = c] \\ & \text{par sécurité parfaite on en déduit :} \\ &= \sum_{m'' \in \mathcal{M}} \Pr_{m' \xleftarrow{d} \mathcal{M}} [m'' = m'] \Pr_{k \leftarrow \Pi.\text{KeyGen}()} [\Pi.\text{Enc}_k(m) = c] \\ &= \sum_{m'' \in \mathcal{M}} \Pr_{m' \xleftarrow{d} \mathcal{M}} [m'' = m'] \Pr_{k \leftarrow \Pi.\text{KeyGen}()} [\Pi.\text{Enc}_k(m) = c] \\ &= \Pr_{k \leftarrow \Pi.\text{KeyGen}()} [\Pi.\text{Enc}_k(m) = c] \left(\sum_{m'' \in \mathcal{M}} \Pr_{m' \xleftarrow{d} \mathcal{M}} [m'' = m'] \right) \\ &= \Pr_{k \leftarrow \Pi.\text{KeyGen}()} [\Pi.\text{Enc}_k(m) = c]. \end{aligned} \tag{2.2}$$

En combinant (2.1), et (2.2), on en déduit ce qu'il fallait démontrer.

On suppose à présent que Π est sécurisé au sens de Shannon.

L'égalité (2.1) reste valide :

$$\begin{aligned} \Pr_{m' \stackrel{d}{\leftarrow} \mathcal{M}, k \leftarrow \Pi.\text{KeyGen}()} [m = m' | \Pi.\text{Enc}_k(m') = c] \\ = \frac{\Pr_{m' \stackrel{d}{\leftarrow} \mathcal{M}} [m = m'] \Pr_{k \leftarrow \Pi.\text{KeyGen}()} [\Pi.\text{Enc}_k(m) = c]}{\Pr_{m' \stackrel{d}{\leftarrow} \mathcal{M}, k \leftarrow \Pi.\text{KeyGen}()} [\Pi.\text{Enc}_k(m') = c]}. \end{aligned}$$

Donc par sécurité de Shannon :

$$\Pr_{m' \stackrel{d}{\leftarrow} \mathcal{M}} [m = m'] = \frac{\Pr_{m' \stackrel{d}{\leftarrow} \mathcal{M}} [m = m'] \Pr_{k \leftarrow \Pi.\text{KeyGen}()} [\Pi.\text{Enc}_k(m) = c]}{\Pr_{m' \stackrel{d}{\leftarrow} \mathcal{M}, k \leftarrow \Pi.\text{KeyGen}()} [\Pi.\text{Enc}_k(m') = c]}.$$

Donc si m est dans le support de d :

$$1 = \frac{\Pr_{k \leftarrow \Pi.\text{KeyGen}()} [\Pi.\text{Enc}_k(m) = c]}{\Pr_{m' \stackrel{d}{\leftarrow} \mathcal{M}, k \leftarrow \Pi.\text{KeyGen}()} [\Pi.\text{Enc}_k(m') = c]}.$$

On pose d la distribution équiprobable sur $\{m, m_2\}$. On a alors :

$$\Pr_{k \leftarrow \Pi.\text{KeyGen}()} [\Pi.\text{Enc}_k(m) = c] = \frac{1}{2} \Pr_{k \leftarrow \Pi.\text{KeyGen}()} [\Pi.\text{Enc}_k(m) = c] + \frac{1}{2} \Pr_{k \leftarrow \Pi.\text{KeyGen}()} [\Pi.\text{Enc}_k(m_2) = c].$$

On en déduit

$$\Pr_{k \leftarrow \Pi.\text{KeyGen}()} [\Pi.\text{Enc}_k(m) = c] = \Pr_{k \leftarrow \Pi.\text{KeyGen}()} [\Pi.\text{Enc}_k(m_2) = c].$$

Et ce pour tout $(m, m_2) \in \mathcal{M}^2$. Donc Π est parfaitement sécurisé. $\square$

Exercice 5 (Théorème de Shannon). *Montrer que dans un cadre où $\mathcal{K} = \mathcal{M} = \mathcal{C}$ sont finis, le chiffrement est parfaitement sécurisé, et correct si et seulement si :*

1. *L'algorithme de génération de clefs consiste à tirer uniformément une clef.*
2. *Il existe pour chaque couple $(m, c) \in \mathcal{M} \times \mathcal{C}$ une unique clef k tel que $\text{Enc}_k(m) = c$.*

Par la suite dans ce chapitre on se restreindra au cas où $\text{KeyGen}(1^\lambda) := \text{Unif}(\{0, 1\}^\lambda)$. En fait, efficacité mise à part, on ne perd pas en généralité avec cette considération si on imagine qu'il s'agit des bits d'aléa de la fonction KeyGen , et que la clef effective peut donc être recalculée à chaque chiffrement¹.

Le Chiffrement de Vernam A présent, nous allons définir dans la figure 2.1 un premier système de chiffrement où les messages sont des chaînes de bits de taille λ : le masque jetable appelé aussi Chiffrement de Vernam ou one-time-pad en anglais. Le concept est simple, on pose $\mathcal{M} = \mathcal{C} = \mathcal{K} = \{0, 1\}^\lambda$ ². L'algorithme de chiffrement, respectivement de déchiffrement, consiste à "xorer" le message, respectivement le cryptogramme, avec une clef qui a été générée uniformément.

Par exemple, si on veut chiffrer le mot binaire 011011 avec la clef 110011, on produit le cryptogramme 101000, qui si il est de nouveau xoré avec la clef redevient le message originel 011011.

Théorème 1. *Le chiffrement de Vernam est un système de chiffrement correct et parfaitement sécurisé.*

Preuve laissée en exercice pour le lecteur. $\square$

1. On peut donc réinterpréter le paramètre de sécurité comme l'entropie de la clef secrète.

2. Le schéma est facilement adaptable en remplaçant $\{0, 1\} = \mathbb{F}_2$ par n'importe quel groupe abélien $\mathbb{Z}/N\mathbb{Z}$. En particulier pour $N = 26$, on se retrouve tout naturellement à la version historique du système appliqué à des mots constitués des lettres de l'alphabet latin.

$\Pi_{MJ} :$ $\text{KeyGen}(1^\lambda) : k \xleftarrow{\$} \{0, 1\}^\lambda$ $\text{Enc}(k, m \in \{0, 1\}^\lambda) :$ Retourner $(k \oplus m)$ $\text{Dec}(k, c) :$ Retourner $(k \oplus c)$	Soit G , un GPA. $\Pi_1 :$ $\text{KeyGen}(1^\lambda) : k \xleftarrow{\$} \{0, 1\}^\lambda$ $\text{Enc}(k, m \in \{0, 1\}^{\ell(\lambda)}) : G(k) \oplus m$ $\text{Dec}(k, c) : \text{Retourner } (G(k) \oplus c)$	Soit F , une FPA. $\Pi_2 :$ $\text{KeyGen}(1^\lambda) : k \xleftarrow{\$} \{0, 1\}^\lambda$ $\text{Enc}(k, m \in \{0, 1\}^{\ell(\lambda)}) :$ $r \xleftarrow{\$} \{0, 1\}^\lambda$ Retourner $(r, F_k(r) \oplus m)$ $\text{Dec}(k, (c_1, c_2)) :$ Retourner $(F_k(c_1) \oplus c_2)$
--	---	--

FIGURE 2.1 – Systèmes de chiffrement

Un problème de taille On pourrait se dire que le problème de la confidentialité est donc complètement résolu avec le système de Vernam. Mais en réalité il suppose une taille de clef aussi grosse que celle du message (donc un partage d'information secrète en amont a priori aussi "difficile" que le partage du secret qu'on cherche à résoudre). Cela peut avoir du sens dans un contexte où on a accès à un canal sécurisé, avant l'envoi du message en lui-même. Par exemple, durant la guerre froide les dirigeants des États-Unis et de l'Union des Républiques socialistes soviétiques communiquaient via une ligne (surnommée le téléphone rouge) où les communications étaient chiffrées avec un chiffrement de Vernam dont les clefs avaient été communiquées en amont par valises diplomatiques (considérées comme canal sécurisé).

Mais dans de très nombreux contextes, cela n'est pas satisfaisant, si je dois chiffrer des données sur un ordinateur collectif, puis garder avec moi un disque dur de la même capacité (au bit près), c'est aussi efficace d'effacer complètement les données après avoir copié les données sur mon disque dur amovible.

Exercice 6. Vous rencontrez quelqu'un qui pratique cette stratégie (chiffrer vernamiquement ses données sur un ordinateur collectif, puis se balader avec la clef de chiffrement sur son disque dur externe), vous lui faites la remarque ci-dessus, et il vous rétorque "Oui, mais si je perds mon disque dur externe, au moins mes données sont toujours sur l'ordinateur collectif, alors qu'avec ce que tu dis, je perds tout".

1. Expliquer à cette personne pourquoi elle se trompe lourdement.
2. L'individu semble convaincu par vos explications, il s'écrit, "J'ai une idée ! Je vais compresser mes données, puis les chiffrer vernamiquement, ainsi la clef sera beaucoup plus petite, et je pourrais la mettre sur une clef usb, ce qui est beaucoup plus pratique à transporter que toutes mes données sur un gros disque dur externe." Expliquer avec pédagogie et sans condescendance déplacée à cette personne que même là, le chiffrement ne lui sert à rien.
3. Trouver quand même un scénario (autre que la perte du disque dur) qui justifie qu'il s'y prenne comme ça.

En fait, on souhaite que les clefs soient de taille plus petite que les messages. Ce qui, si on évacue les questions de compression d'information (cf question 2 de l'exercice 6) implique par rapport aux cardinaux : $|\mathcal{M}| > |\mathcal{K}|$. Malheureusement, on peut montrer le résultat suivant :

Théorème 2. Si le système est correct et parfaitement sécurisé on a $|\mathcal{M}| \leq |\mathcal{K}|$.

Lemme 1. Si le système est correct alors on peut supposer sans perdre de généralités que $\Pi.\text{Dec}_k$ est déterministe.

Soit $c \in \mathcal{C}$.

Premier cas, il existe $m_0 \in \mathcal{M}$, et un aléa r tel que $\Pi.\text{Enc}_k(m_0; r) = c$. Par correction $\Pr[\Pi.\text{Dec}_k(c) = m_0] = 1$. Donc on utilise pas d'aléa dans ce cas.

Second cas, $c \notin \Pi.\text{Enc}_k(\mathcal{M})$, on peut décider de déterminer l'algorithme sans avoir d'impact ni sur la correction ni sur la sécurité parfaite (qui ne dit rien sur $\Pi.\text{Dec}$). On en déduit qu'on peut considérer que $\Pi.\text{Dec}$ est déterministe. $\square$

Sans perdre de généralité, on peut supposer qu'il existe $(m_0, c) \in \mathcal{M} \times \mathcal{C}$, tel que

$$\Pr_{k \leftarrow \Pi.\text{KeyGen}} [\Pi.\text{Enc}_k(m_0) = c] \neq 0.$$

Et comme le système est parfaitement sécurisé, on en déduit :

$$\forall m \in \mathcal{M} : \Pr_{k \leftarrow \Pi.\text{KeyGen}} [\Pi.\text{Enc}_k(m) = c] \neq 0.$$

Ce qui implique $\forall m \in \mathcal{M} : \Pr_{k \leftarrow \Pi.\text{KeyGen}} [\Pi.\text{Dec}_k(c) = m] = \Pr_{k \leftarrow \Pi.\text{KeyGen}} [\Pi.\text{Enc}_k(m_0) = c] \neq 0$.

Donc on en déduit $\mathcal{M} \subset \{\exists k \in \mathcal{K} : c \in \{\Pi.\text{Enc}_k(m)\}_{k \in \mathcal{K}}\}$.

Donc par correction $\mathcal{M} \subset \{\exists k \in \mathcal{K} : c \in \{\Pi.\text{Dec}_k(c)\}_{k \in \mathcal{K}}\} = \Pi.\text{Dec}(\mathcal{K}, c)$. Comme $\Pi.\text{Dec}$ est déterministe, on peut borner le cardinal de $\Pi.\text{Dec}(\mathcal{K}, c)$, par $|\mathcal{K}|$. On en déduit donc ce qu'il fallait démontrer : $|\mathcal{M}| \leq |\mathcal{K}|$. $\square$

Le théorème 2 nous contraint donc à abandonner le critère de la sécurité parfaite pour définir la confidentialité.

Un autre de problème de taille On peut réécrire la sécurité parfaite sous une forme de jeu de sécurité avec l'exercice suivant.

Exercice 7. Montrer que la définition 29 est équivalente à dire que pour tout couples de machines de Turing $\mathcal{A} = (\mathcal{A}_1, \mathcal{A}_2)$,

$$\Pr [\text{PrivK}_{\mathcal{A}, \Pi}^{\text{eav0}}() = 1] = \frac{1}{2}$$

où $\text{PrivK}_{\mathcal{A}, \Pi}^{\text{eav0}}$ est le jeu de l'indiscrétion (eavsdropper en anglais) non paramétré (figure 2.2).

Avant d'aller plus loin on remarque que si on s'intéresse au cas où l'ensemble des messages est infini, on se rend compte des limitations intrinsèques de la confidentialité des systèmes de chiffrement. Pour comprendre l'intuition, il suffit de remarquer que si je veux savoir si deux personnes se sont transmis un film en 3D très lourd, ou un fichier de texte d'un haïku, je peux savoir dans quel cas on est en regardant le poids des données transmises, et ce même si les données sont chiffrées, car on s'attend à ce qu'un système de chiffrement conserve la taille des données. L'exercice 11 à la fin de ce chapitre montre en quoi cette limitation est structurelle dans le cas où la taille de l'ensemble des messages est infinie³.

2.1.2 Sécurité calculatoire : Sécurité contre l'indiscrétion

La sécurité parfaite nous apparaît inaccessible suite au théorème 2, mais heureusement, comme dans le chapitre 1, on peut partir du principe qu'on cherche avant tout à se protéger vis à vis d'un adversaire calculatoirement borné en tolérant qu'il réussisse avec un succès très faible à "casser le système". On va donc réutiliser les notions de PPT, et de fonctions négligeables vues dans le précédent chapitre.

Et donc il est nécessaire de redéfinir les schémas de chiffrement en introduisant le paramètre de sécurité :

Définition 30. Un schéma de chiffrement est un triplet d'algorithmes PPT : $(\text{KeyGen}, \text{Enc}, \text{Dec})$.

$$\begin{aligned} \text{KeyGen} &: \left\{ \{0, 1\}^\lambda \right\}_{\lambda \in \mathbb{N}} \rightarrow \mathcal{K} \\ \text{Enc} &: \mathcal{K} \times \mathcal{M} \rightarrow \mathcal{C} \\ \text{Dec} &: \mathcal{K} \times \mathcal{C} \rightarrow \mathcal{M}. \end{aligned}$$

On se rend compte qu'on peut alors transformer le jeu $\text{PrivK}^{\text{eav0}}$ en le paramétrant ce qui donne la définition suivante :

Définition 31. Soit Π , un schéma de chiffrement. Pour tout PPT $\mathcal{A}$, on appelle l'avantage de $\mathcal{A}$, la valeur $\text{Adv}_{\mathcal{A}, \Pi}^{\text{eav}}(\lambda) := \left| \frac{1}{2} - \Pr[\text{PrivK}_{\mathcal{A}, \Pi}^{\text{eav}}(\lambda) = 1] \right|$. On dit que le schéma Π est eav-sécurisé si et seulement si pour tout PPT $\mathcal{A}$, $\text{Adv}_{\mathcal{A}, \Pi}^{\text{eav}}(\lambda)$ est négligeable, où $\text{PrivK}_{\mathcal{A}, \Pi}^{\text{eav}}$ est le jeu de l'indiscrétion (figure 2.2).

On remarque que comme l'adversaire est en deux étapes, et que son temps de calcul est borné, l'adversaire peut fabriquer une mémoire (représentée par la chaîne de bits η) lors de la première étape de calcul (c'est à dire l'étape qui correspond au choix du contexte), puis ensuite utiliser cette mémoire dans la deuxième étape de calcul (ce qui explique que η est prise en entrée par $\mathcal{A}_2$).

On remarque également que ce jeu ne cherche à garantir la confidentialité à taille des données connue.

Dans la suite on pourra parfois utiliser une autre définition :

3. Et même dans le cas où l'ensemble des messages est fini, quand on souhaite un système de chiffrement efficace avec $|\text{Enc}(m)| \approx |m|$, l'accès à une donnée chiffrée divulgue des informations sur la taille de la donnée en clair.

$\text{PrivK}_{\mathcal{A},\Pi}^{\text{eav}0}() :$ $k \leftarrow \Pi.\text{KeyGen}()$ $(m_0, m_1) \leftarrow \mathcal{A}_1()$ $b \xleftarrow{\$} \{0, 1\}$ $b' \leftarrow \mathcal{A}_2(\Pi.\text{Enc}_k(m_b))$ Retourner $b \stackrel{?}{=} b'$	$\text{PrivK}_{\mathcal{A},\Pi}^{\text{eav}}(\lambda) :$ $k \leftarrow \Pi.\text{KeyGen}(1^\lambda)$ $(m_0, m_1, \eta) \leftarrow \mathcal{A}_1(1^\lambda)$ $b \xleftarrow{\$} \{0, 1\}$ $b' \leftarrow \mathcal{A}_2(\Pi.\text{Enc}_k(m_b), \eta)$ Si $ m_0 \neq m_1 $: Retourner b Sinon Retourner $b \stackrel{?}{=} b'$	$\text{PrivK}_{\mathcal{A},\Pi}^{\text{eav}2}(\lambda, b) :$ $k \leftarrow \Pi.\text{KeyGen}(1^\lambda)$ $(m_0, m_1, \eta) \leftarrow \mathcal{A}_1(1^\lambda)$ $b' \leftarrow \mathcal{A}_2(\Pi.\text{Enc}_k(m_b), \eta)$ Si $ m_0 \neq m_1 $: Retourner $\perp$ Sinon Retourner b'
---	--	--

FIGURE 2.2 – Jeux de sécurité de l’indiscrétion : une version non paramétrée, puis deux versions paramétrées

Définition 32. Soit Π , un schéma de chiffrement. On dit que le schéma Π est eav-sécurisé si et seulement si pour tout PPT $\mathcal{A}$:

$$|\Pr[\text{PrivK}_{\mathcal{A},\Pi}^{\text{eav}2}(\lambda, 0) = 1] - \Pr[\text{PrivK}_{\mathcal{A},\Pi}^{\text{eav}2}(\lambda, 1) = 1]|$$

est négligeable, où $\text{PrivK}_{\mathcal{A},\Pi}^{\text{eav}2}$ est la deuxième version du jeu de l’indiscrétion (figure 2.2).

Exercice 8. Montrer que les définitions 31, et 32 sont équivalentes.

Preuve laissée en exercice au lecteur. $\square$

Tout d’abord, on peut remarquer avec la propriété suivante que cette définition est plus faible que la sécurité parfaite :

Propriété 3. Un schéma de chiffrement parfaitement sécurisé est eav-sécurisé⁴.

Preuve laissée en exercice pour le lecteur. $\square$

Corollaire 1. Le masque jetable est eav-sécurisé.

A présent, cherchons à construire un schéma qui vérifie cette propriété tout en ayant des tailles de clefs strictement plus petites que les tailles de messages. Dans l’idée on va vouloir reprendre le masque jetable, mais avec un moyen de “compresser” la clef. A priori ce n’est pas possible de compresser sans perte une clef ayant autant d’entropie que sa taille. Si on raisonne dans l’autre sens, il s’agirait de simuler une clef de $\lambda + 1$ bits d’entropie qui en réalité ne contient que λ bits d’entropie. Ceci nous fait penser aux générateurs pseudo-aléatoires largement utilisés depuis longtemps dans les simulations de phénomènes physiques, ces algorithmes sont censés prendre en entrée des chaînes d’“aléa réel” pour produire efficacement une séquence importante de bits qui “ressembleront” à des bits complètement aléatoires qui seraient beaucoup plus coûteux et compliqués à produire.

On pourrait penser à reprendre tel quels ces générateur pseudo-aléatoires, mais la notion de “ressemblance à de l’aléa”, n’a pas été pensée dans un contexte de sécurité⁵ ;

On va donc devoir recourir à une définition propre à un contexte de sécurité :

Définition 33 (Générateur pseudo aléatoire). Soit $\ell : \mathbb{N} \rightarrow \mathbb{N}$, une fonction polynomiale. Soit G une fonction DPT telle que pour $\lambda \in \mathbb{N}$, pour tout $s \in \{0, 1\}^\lambda$: $G(s) \in \{0, 1\}^{\ell(\lambda)}$. On dit que G est pseudo-aléatoire si et seulement si elle vérifie les deux propriétés suivantes :

Expansion : $\forall \lambda \in \mathbb{N} : |\ell(\lambda)| > |\lambda|$.

Pseudo-aléa : Pour tout PPT D :

$$\left| \Pr_{s \xleftarrow{\$} \{0,1\}^\lambda} [D(G(s)) = 1] - \Pr_{x \xleftarrow{\$} \{0,1\}^{\ell(\lambda)}} [D(x) = 1] \right|$$

est négligeable en λ .

4. On paramétrera canoniquement le schéma en redéfinissant $\forall \lambda \in \mathbb{N} : \text{KeyGen}(1^\lambda) := \text{KeyGen}()$.

5. En général un GPA est considéré comme bon si il passe un certain nombre de tests statistiques “basiques”.

$\mathcal{D}_{\mathcal{A}}(x) :$ On suppose λ calculable à partir de $ x $ ^a $(m_0, m_1, \eta) \leftarrow \mathcal{A}_1(1^\lambda)$ $b \xleftarrow{\$} \{0, 1\}$ $b' \leftarrow \mathcal{A}_2(x \oplus m_b, \eta)$ Si $ m_0 \neq m_1 $ Retourner b Sinon Retourner $b \stackrel{?}{=} b'$	$\text{PrivK}_{\mathcal{A}, \Pi_1}^{\text{eav}}(\lambda) :$ $k \leftarrow \{0, 1\}^\lambda$ $(m_0, m_1, \eta) \leftarrow \mathcal{A}_1(1^\lambda)$ $b \xleftarrow{\$} \{0, 1\}$ $b' \leftarrow \mathcal{A}_2(G(k) \oplus m_b, \eta)$ Si $ m_0 \neq m_1 $: Retourner b Sinon Retourner $b \stackrel{?}{=} b'$	$\mathcal{D}_{\mathcal{A}}^{\text{random}}(1^\lambda) :$ $b \xleftarrow{\$} \{0, 1\}$ $x \xleftarrow{\$} \{0, 1\}^{\ell(\lambda)}$ $(m_0, m_1, \eta) \leftarrow \mathcal{A}_1(1^\lambda)$ $b' \leftarrow \mathcal{A}_2(x \oplus m_b, \eta)$ Si $ m_0 \neq m_1 $: Retourner b Sinon Retourner $b \stackrel{?}{=} b'$
---	---	--

FIGURE 2.3 – Attaquant $\mathcal{D}$ contre le GPA G , jeu de l'indiscrétion pour Π_1 , et $\mathcal{D}^{\text{random}}$ un algorithme qui simule $\mathcal{D}$ qui prend du “vrai aléa” en entrée.

Dans la suite de ce cours lorsqu'on fera référence à un GPA (Générateur pseudo aléatoire), on fera référence à cette définition et non à la définition "hors cryptographie". Grâce à cette fonctionnalité, on va donc, en utilisant seulement x une chaîne de λ bits d'entropie, pouvoir générer une clef de masque jetable $G(x)$ de $\ell(\lambda)$ bits tout en conservant la sécurité de l'indiscrétion avec le schéma Π_1 :

Théorème 3. *Si G est un GPA, alors Π_1 (figure 2.1) est eav-sécurisé.*

On définit le distingueur $\mathcal{D}$ dans la figure 2.3. On remarque d'abord que si l'entrée x est égal à $G(k)$ avec k tiré uniformément dans $\{0, 1\}^\lambda$. Alors la distribution $\mathcal{D}(x)$ est identique à celle $\mathcal{D}_{\mathcal{A}}^{\text{random}}(1^\lambda)$.

Quand on étudie en détail cet algorithme, on se rend compte que b est indépendant de x , donc de $x \oplus m_b$ (car x suit la distribution uniforme), donc de b' , ainsi $b \stackrel{?}{=} b'$ est une variable aléatoire de Bernoulli de paramètre $\frac{1}{2}$.

On en déduit donc : $\Pr_{x \xleftarrow{\$} \{0, 1\}^{\ell(\lambda)}} [\mathcal{D}_{\mathcal{A}}(x) = 0] = \frac{1}{2}$.

D'un autre côté on remarque que la distribution de $\mathcal{D}_{\mathcal{A}}(G(s))$, avec s tiré uniformément dans $\{0, 1\}^\lambda$, est identique à celle de $\text{PrivK}_{\mathcal{A}, \Pi_1}^{\text{eav}}(\lambda)$.

Donc on en déduit que $|\Pr[\text{PrivK}_{\mathcal{A}, \Pi_1}^{\text{eav}}(\lambda) = 0] - \frac{1}{2}|$ est égal à

$$\left| \Pr_{s \xleftarrow{\$} \{0, 1\}^\lambda} [\mathcal{D}_{\mathcal{A}}(G(s)) = 0] - \Pr_{x \xleftarrow{\$} \{0, 1\}^{\ell(\lambda)}} [\mathcal{D}_{\mathcal{A}}(x) = 0] \right|. \quad (2.3)$$

On remarque que comme $\mathcal{A}_1$, et $\mathcal{A}_2$ sont des algorithmes *PPT* par hypothèse, alors $\mathcal{D}$ est aussi un algorithme *PPT*. Or si G est un GPA, (2.3) est négligeable en λ . On en déduit donc

$$\left| \Pr[\text{PrivK}_{\mathcal{A}, \Pi_1}^{\text{eav}}(\lambda) = 0] - \frac{1}{2} \right| = \text{neg}(\lambda).$$

Ce qu'il fallait démontrer. □

Les hypothèses cryptographiques : un des fondements de la cryptologie moderne Nous avons construit le premier schéma de chiffrement “non-trivial” en terme d'efficacité. Pour passer ce cap d'efficacité, la sécurité de ce schéma doit non seulement reposer sur une preuve mathématique, mais aussi sur une *hypothèse cryptographique*, qui sera bien souvent réduit à l'état de conjecture : “ G est un GPA”. Ainsi on ne garantit plus stricto sensu que la sécurité de schéma restera inviolée, mais que si elle devait être brisée alors la conjecture concernant G serait cassée. Et si cela devait arriver cela ne rendrait pas notre schéma inopérant pour autant, il “suffirait” alors de l'utiliser avec un nouveau G qui vérifierait bien les propriétés des GPA.

Comment choisir un bon candidat pour une hypothèse cryptographique ? Pour choisir un bon candidat, on doit sortir des mathématiques pures, et renouer avec le principe de Kerckhoffs⁶. Les potentiels candidats (par exemple pour un GPA) doivent donc être présentés publiquement et le plus largement diffusés de manière à ce que le plus possible de gens cherchent à “casser” la conjecture “le candidat est un GPA”, par exemple en trouvant un algorithme PPT D tel que la grandeur $\left| \Pr_{x \leftarrow \{0,1\}^\lambda} [D(G(x)) = 1] - \Pr_{x \leftarrow \{0,1\}^{\ell(\lambda)}} [D(x) = 1] \right|$ soit minorée par une fraction rationnelle en λ . Ce paradigme a eu l’effet concret d’intégrer très fortement la cryptologie (aussi bien la cryptographie que la cryptanalyse) à la sphère académique, alors qu’elle était auparavant la chasse gardée des militaires.

2.1.3 Sécurité calculatoire : Sécurité contre les attaques à clairs choisis

Dans cette partie, nous allons chercher à renforcer notre modèle de sécurité (avec pour ambition de ne pas retomber dans la rigidité de la sécurité parfaite). La faiblesse principale des deux précédents modèles (aussi bien la sécurité parfaite que la modèle de l’indiscrétion) est dans la non réutilisation de la clef : Si on chiffre ne serait-ce qu’un deuxième message avec la même clef ces propriétés ne garantissent absolument rien sur la confidentialité des messages (y-compris du premier message chiffré).

On peut chercher à construire un modèle de sécurité dans l’exercice suivant :

- Exercice 9.**
1. Construire un jeu de sécurité (avec la définition associée) qui permet de capturer un scénario où l’adversaire peut chiffrer deux messages.
 2. A présent, on souhaite renforcer le modèle de la question précédente en laissant à l’adversaire la possibilité de choisir le second message en fonction du chiffré du premier.
 3. Généraliser les jeux des exercices précédents au contexte où on a n messages au lieu de 2 (avec un paramètre du jeu traité de manière analogue au paramètre de sécurité).

Le dernier modèle de l’exercice précédent est intéressant, mais il a l’inconvénient d’avoir un paramètre (ce qu’on veut éviter au maximum), et d’être relativement difficile à lire. On va donc utiliser le paradigme de “renforcement par simplification” en autorisant l’adversaire à demander autant de messages qu’il le souhaite. Pour modéliser cette capacité, on va revenir à la notion d’oracles vue dans le chapitre 1. Ce qui revient donc au jeu de l’attaque à texte clairs choisis, $\text{PrivK}^{\text{cpa}}$ de la figure 2.4 (CPA est l’acronyme de *Chosen Plaintext Attack*), et à la définition associée :

Définition 34. Soit Π , un schéma de chiffrement. On dit que le schéma Π est sécurisé contre les attaques à clairs choisis si et seulement si pour tout PPT $\mathcal{A}$, la valeur

$$\text{Adv}_{\mathcal{A}, \Pi}^{\text{cpa}}(\lambda) := \left| \Pr[\text{PrivK}_{\mathcal{A}, \Pi}^{\text{cpa}}(\lambda) = 1] - \frac{1}{2} \right|$$

est négligeable en λ .

Dans un exercice ultérieur on montrera pourquoi ce modèle est strictement plus puissant que ceux vus à l’exercice 9.

Construction du schéma On remarque que contrairement au modèle de sécurité de la partie précédente, étant donnée que la clef a vocation à être réutilisée plusieurs fois, on ne va plus chercher à que cette dernière soit plus petite que le message envoyé.

Pour vérifier une telle propriété de sécurité, il faudrait pouvoir générer un masque de Vernam pour chaque message que l’adversaire demanderait à l’oracle. Il apparaît clairement que les GPA ne vont pas suffire pour pouvoir générer autant de bits pseudo-aléatoire. On va donc recourir à une primitive fondamentale plus forte, les fonctions pseudo-aléatoires qui avec une unique clef peuvent former du pseudo-aléa à volonté (tant que cette volonté reste polynomialement bornée).

6. Le principe de Kerckhoffs préconise que toutes les méthodes utilisées concernant la confidentialité d’un système de communication n’ont pas à être supposées secrètes, et que seule la clef (une donnée facile à générer, et à communiquer) doit être secrète.

$\text{PrivK}_{\mathcal{A}, \Pi}^{\text{cpa}}(\lambda) :$ $k \leftarrow \Pi.\text{KeyGen}(1^\lambda)$ $(m_0, m_1, \eta) \leftarrow \mathcal{A}_1^{\Pi.\text{Enc}_k(\cdot)}(1^\lambda)$ $b \xleftarrow{\$} \{0, 1\}$ $b' \leftarrow \mathcal{A}_2^{\Pi.\text{Enc}_k(\cdot)}(\Pi.\text{Enc}_k(m_b))$ Si $ m_0 \neq m_1 $ Retourner b Sinon Retourner $b \stackrel{?}{=} b'$	$\text{PrivK}_{\mathcal{A}, \Pi}^{\text{cca}}(\lambda) :$ $k \leftarrow \Pi.\text{KeyGen}(1^\lambda)$ $(m_0, m_1, \eta) \leftarrow \mathcal{A}_1^{\Pi.\text{Enc}_k(\cdot), \Pi.\text{Dec}_k(\cdot)}(1^\lambda)$ $b \xleftarrow{\$} \{0, 1\}$ $c^* \leftarrow \Pi.\text{Enc}_k(m_b)$ $b' \leftarrow \mathcal{A}_2^{\Pi.\text{Enc}_k(\cdot), \text{ODec}(\cdot)}(c^*)$ Si $ m_0 \neq m_1 $ Retourner b Sinon Retourner $b \stackrel{?}{=} b'$	$\text{ODec}(c) :$ Si $c \stackrel{?}{=} c^*$ Retourner $\perp$ Sinon Retourner $\Pi.\text{Dec}(k, c)$
---	--	---

FIGURE 2.4 – Jeu de sécurité avec adversaire ayant accès aux oracles de chiffrement, puis accès aux oracles de chiffrement et de déchiffrement

$\mathcal{D}_A^O(1^\lambda) :$ $(m_0, m_1, \eta) \leftarrow \mathcal{A}_1^{O\text{Enc}(\cdot)}(1^\lambda)$ $b \xleftarrow{\$} \{0, 1\}$ $r \xleftarrow{\$} \{0, 1\}^\lambda$ $b' \leftarrow \mathcal{A}_2^{O\text{Enc}(\cdot)}((r, O(r) \oplus m_b), \eta)$ Si $ m_0 \neq m_1 $ Retourner b Sinon Retourner $b \stackrel{?}{=} b'$	$O\text{Enc}(x) :$ Sinon $r \xleftarrow{\$} \{0, 1\}^\lambda$ Retourner $(r, O(r) \oplus x)$	$\text{PrivK}_{\mathcal{A}, \Pi_2}^{\text{cpa}}(\lambda) :$ $k \leftarrow \{0, 1\}^\lambda$ $(m_0, m_1, \eta) \leftarrow \mathcal{A}_1^{\Pi.\text{Enc}_k(\cdot)}(1^\lambda)$ $r \xleftarrow{\$} \{0, 1\}^\lambda$ $b \xleftarrow{\$} \{0, 1\}$ $b' \leftarrow \mathcal{A}_2^{\text{Enc}_k(\cdot)}((r, F_k(r) \oplus m_b), \eta)$ Si $ m_0 \neq m_1 $: Retourner b Sinon Retourner $b \stackrel{?}{=} b'$
--	--	--

FIGURE 2.5 – Attaquant $\mathcal{D}$ contre la FPA F , et jeu de l'indiscrétion pour Π_2 .

Définition 35. [Fonction pseudo-aléatoire] Soit $F : \{0, 1\}^* \times \{0, 1\}^* \rightarrow \{0, 1\}^*$, déterministe, polynomialement bornée et vérifie $\forall k : F(k, \cdot)$ est une fonction qui préserve la taille de l'entrée. On dit que F est pseudo-aléatoire si pour tout adversaire PPT $\mathcal{A}$:

$$\left| \Pr_{k \xleftarrow{\$} \{0, 1\}^\lambda} \left(\mathcal{A}^{F_k(\cdot)}(1^\lambda) = 1 \right) - \Pr \left(\mathcal{A}^{f(\cdot)}(1^\lambda) = 1 \right) \right| \leq \text{neg}(\lambda)$$

où F_k est la restriction de F à $\{k\} \times \{0, 1\}^\lambda$ et $f : \{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda$ est complètement aléatoire. On peut également réécrire la contrainte avec F_k et f permutations, et à ce moment là on rajoute les oracles F_k^{-1}, f^{-1} (et on parle alors de permutation pseudo-aléatoire (PPA)).

Grâce à une telle primitive fondamentale on va pouvoir générer un nouveau “masque jetable” $F_k(r)$ pour chaque chiffrement de message avec r une chaîne aléatoire qui sera accolée au chiffré de Vernam :

Théorème 4. Si F est une FPA, Π_2 (figure 2.1) est CPA sécurisé.

On définit le distingueur $\mathcal{D}$ dans la figure 2.5. On remarque d'abord que la distribution de $\mathcal{D}_A^{F_k}(1^\lambda)$ avec $k \xleftarrow{\$} \{0, 1\}^\lambda$ est identique à celle de $\text{PrivK}_{\mathcal{A}, \Pi_2}^{\text{cpa}}(\lambda)$ (cf figure 2.5).

Regardons à présent la distribution de $\mathcal{D}_A^f(1^\lambda)$ avec $f \xleftarrow{\$} (\{0, 1\}^\lambda)^{\{0, 1\}^\lambda}$. En remarquant que le choix de f revient à choisir $f(y) \in \{0, 1\}^\lambda$ pour tout $y \in \{0, 1\}^\lambda$. En choisissant ces $f(y)$ “à la volée”, on ne change pas la distribution. Ainsi cette distribution est identique à celle de $\mathcal{D}_A^{(2)}$, et donc à celle de $\mathcal{D}^{(3)}$ (où pour cet algorithme, on a seulement décomposé l'appel à $O\text{Alea}$ pour l'entrée de $\mathcal{A}_2$).

L'algorithme $\mathcal{D}^{(4)}(1^\lambda)$ a une distribution identique à celle de $\mathcal{D}^{(3)}(1^\lambda)$ sauf si l'évènement $E := (r, \cdot) \in Q$ arrive.

Regardons à présent la distribution de $\mathcal{D}^{(4)}(1^\lambda)$. On remarque que x est indépendant de b , et donc comme x suit la distribution uniforme $x \oplus m_b$ est indépendant de b , donc b' est indépendant de b . Ainsi aussi bien b que $b \stackrel{?}{=} b'$ suivent toutes les deux indépendamment des lois de Bernoulli de paramètre $\frac{1}{2}$.

$\mathcal{D}_A^{(2)}(1^\lambda) :$ $Q := \emptyset$ $(m_0, m_1, \eta) \leftarrow \mathcal{A}_1^{OAlea(\cdot)}(1^\lambda)$ $b \xleftarrow{\$} \{0, 1\}$ $b' \leftarrow \mathcal{A}_2(OAlea(m_b), \eta)$ Si $ m_0 \neq m_1 $ Retourner b Sinon Retourner $b \stackrel{?}{=} b'$	$\mathcal{D}_A^{(3)}(1^\lambda) :$ $Q := \emptyset$ $(m_0, m_1, \eta) \leftarrow \mathcal{A}_1^{OAlea(\cdot)}(1^\lambda)$ $b \xleftarrow{\$} \{0, 1\}$ $r, x \xleftarrow{\$} \{0, 1\}^\lambda$ $Q := Q \cup \{(r, x)\}$ $b' \leftarrow \mathcal{A}_2^{OAlea(\cdot)}((r, x \oplus m_b), \eta)$ Si $ m_0 \neq m_1 $ Retourner b Sinon Retourner $b \stackrel{?}{=} b'$	$OAlea(m) :$ $x \xleftarrow{\$} \{0, 1\}^\lambda$ Si $\exists (x, y) \in Q$ Retourner $(x, y \oplus m)$ Sinon $y \xleftarrow{\$} \{0, 1\}^\lambda$ $Q := Q \cup \{(x, y)\}$ Retourner $(x, y \oplus m)$
---	---	---

FIGURE 2.6 – Algorithmes $\mathcal{D}^{(2)}$, et $\mathcal{D}^{(3)}$.

$\mathcal{D}_A^{(4)}(1^\lambda) :$ $Q := \emptyset$ $(m_0, m_1, \eta) \leftarrow \mathcal{A}_1^{OAlea(\cdot)}(1^\lambda)$ $b \xleftarrow{\$} \{0, 1\}$ $r, x \xleftarrow{\$} \{0, 1\}^\lambda$ $b' \leftarrow \mathcal{A}_2^{OAlea(\cdot)}((r, x \oplus m_b), \eta)$ Si $ m_0 \neq m_1 \wedge (r, x) \in Q$ Retourner b Sinon Retourner $b \stackrel{?}{=} b'$	$OAlea(m) :$ $x \xleftarrow{\$} \{0, 1\}^\lambda$ Si $\exists (x, y) \in Q$ retourner $(x, y \oplus m)$ Sinon $y \xleftarrow{\$} \{0, 1\}^\lambda$ $Q := Q \cup \{(x, y)\}$ Retourner $(x, y \oplus m)$
---	--

FIGURE 2.7 – Algorithme $\mathcal{D}^{(4)}$.

Donc on en déduit que

$$\begin{aligned}
 & \left| \Pr[\text{PrivK}_{\mathcal{A}, \Pi_1}^{\text{cpa}}(\lambda) = 0] - \frac{1}{2} \right| \\
 &= \left| \Pr_{k \xleftarrow{\$} \{0, 1\}^\lambda} \left[\mathcal{D}_A^{F_k(\cdot)}(1^\lambda) = 0 \right] - \Pr \left[\mathcal{D}_A^{(4)}(1^\lambda) = 0 \right] \right|
 \end{aligned} \tag{2.4}$$

Regardons le deuxième terme dans le détail, en posant $E :=$ le x tiré au moment du calcul du cryptogramme challenge a déjà été tiré lors des appels à $OAlea$:

$$\begin{aligned}
 & \Pr \left[\mathcal{D}_A^{(4)}(1^\lambda) = 0 \right] \\
 &= \Pr \left[\mathcal{D}_A^{(4)}(1^\lambda) = 0 | E \right] \Pr[E] - \Pr \left[\mathcal{D}_A^{(4)}(1^\lambda) = 0 | \bar{E} \right] \Pr[\bar{E}] \\
 &= \Pr \left[\mathcal{D}_A^{(4)}(1^\lambda) = 0 | E \right] \Pr[E] + \Pr \left[\mathcal{D}_A^{(3)}(1^\lambda) = 0 | \bar{E} \right] (1 - \Pr[E]) \\
 &= \Pr \left[\mathcal{D}_A^{(4)}(1^\lambda) = 0 | E \right] \Pr[E] + \Pr \left[\mathcal{D}_A^{(3)}(1^\lambda) = 0 \right] - \Pr \left[\mathcal{D}_A^{(3)}(1^\lambda) = 0 | E \right] \Pr[E] \\
 &= \Pr \left[\mathcal{D}_A^{(3)}(1^\lambda) = 0 \right] + \Pr[E] \left(\Pr \left[\mathcal{D}_A^{(4)}(1^\lambda) = 0 | E \right] - \Pr \left[\mathcal{D}_A^{(3)}(1^\lambda) = 0 | E \right] \right)
 \end{aligned} \tag{2.5}$$

On remarque que $\Pr[E] \leq \frac{q}{2^\lambda}$, où q représente le nombre de requêtes total de $\mathcal{A}$. On déduit donc en remarquant que $\Pr \left[\mathcal{D}_A^{(4)}(1^\lambda) = 0 | E \right]$ et $\Pr \left[\mathcal{D}_A^{(3)}(1^\lambda) = 0 | E \right]$ sont bornés par 1 à partir de (2.5) :

$$\left| \Pr \left[\mathcal{D}_A^{(4)}(1^\lambda) = 0 \right] - \Pr \left[\mathcal{D}_A^{(3)}(1^\lambda) = 0 \right] \right| \leq \frac{q}{2^\lambda}. \tag{2.6}$$

Donc en combinant le fait que $\mathcal{D}^{(3)}$ et $\mathcal{D}^f(\cdot)$ (avec f tirée uniformément aléatoire parmi les fonctions

de $\{0, 1\}^\lambda \rightarrow \{0, 1\}^\lambda$) suivent la même distribution et (2.6) on déduit que :

$$\left| \Pr_{f \leftarrow \mathbb{S}_{(\{0,1\}^\lambda)^{\{0,1\}^\lambda}}} \left[\mathcal{D}_{\mathcal{A}}^{f(\cdot)}(1^\lambda) = 0 \right] - \Pr \left[\mathcal{D}_{\mathcal{A}}^{(4)}(1^\lambda) = 0 \right] \right| \leq \frac{q}{2^\lambda}. \quad (2.7)$$

Et par inégalité triangulaire on a aussi :

$$\begin{aligned} & \left| \Pr_{k \leftarrow \mathbb{S}_{\{0,1\}^\lambda}} \left[\mathcal{D}_{\mathcal{A}}^{F_k(\cdot)}(1^\lambda) = 0 \right] - \Pr \left[\mathcal{D}_{\mathcal{A}}^{(4)}(1^\lambda) = 0 \right] \right| \\ & \leq \left| \Pr_{f \leftarrow \mathbb{S}_{(\{0,1\}^\lambda)^{\{0,1\}^\lambda}}} \left[\mathcal{D}_{\mathcal{A}}^{f(\cdot)}(1^\lambda) = 0 \right] - \Pr \left[\mathcal{D}_{\mathcal{A}}^{(4)}(1^\lambda) = 0 \right] \right| \\ & \quad + \left| \Pr_{f \leftarrow \mathbb{S}_{(\{0,1\}^\lambda)^{\{0,1\}^\lambda}}} \left[\mathcal{D}_{\mathcal{A}}^{f(\cdot)}(1^\lambda) = 0 \right] - \Pr_{k \leftarrow \mathbb{S}_{\{0,1\}^\lambda}} \left[\mathcal{D}_{\mathcal{A}}^{F_k(\cdot)}(1^\lambda) = 0 \right] \right|. \end{aligned} \quad (2.8)$$

En appliquant (2.4), et (2.7) à (2.8), on en déduit :

$$\begin{aligned} & \left| \Pr[\text{PrivK}_{\mathcal{A}, \Pi_1}^{\text{cpa}}(\lambda) = 0] - \frac{1}{2} \right| \\ & \leq \frac{q}{2^\lambda} + \left| \Pr_{f \leftarrow \mathbb{S}_{(\{0,1\}^\lambda)^{\{0,1\}^\lambda}}} \left[\mathcal{D}_{\mathcal{A}}^{f(\cdot)}(1^\lambda) = 0 \right] - \Pr_{k \leftarrow \mathbb{S}_{\{0,1\}^\lambda}} \left[\mathcal{D}_{\mathcal{A}}^{F_k(\cdot)}(1^\lambda) = 0 \right] \right|. \end{aligned} \quad (2.9)$$

On remarque que comme $\mathcal{A}_1$, et $\mathcal{A}_2$ sont des algorithmes *PPT* par hypothèse, alors $\frac{q}{2^\lambda}$ est négligeable en λ et $\mathcal{D}$ est aussi un algorithme *PPT*. Or si F est une FPA :

$$\left| \Pr_{f \leftarrow \mathbb{S}_{(\{0,1\}^\lambda)^{\{0,1\}^\lambda}}} \left[\mathcal{D}_{\mathcal{A}}^{f(\cdot)}(1^\lambda) = 0 \right] - \Pr_{k \leftarrow \mathbb{S}_{\{0,1\}^\lambda}} \left[\mathcal{D}_{\mathcal{A}}^{F_k(\cdot)}(1^\lambda) = 0 \right] \right| = \text{neg}(\lambda).$$

On en déduit donc de (2.9) :

$$\left| \Pr[\text{PrivK}_{\mathcal{A}, \Pi_1}^{\text{cpa}}(\lambda) = 0] - \frac{1}{2} \right| = \text{neg}(\lambda).$$

Ce qu'il fallait démontrer. □

2.1.4 Sécurité calculatoire : Sécurité contre les attaques à chiffrés choisis

Assez naturellement, on va chercher à étendre la notion de sécurité en imaginant un adversaire qui non seulement à accès à des oracles de chiffrement, mais aussi de déchiffrement. Mais si on ne fait pas attention, lorsqu'on définit le jeu de sécurité on peut tomber dans *l'écueil du jeu structurellement trop fort* : C'est à dire que si l'adversaire demande à l'oracle de déchiffrement de déchiffrer c^* , il va très facilement se rendre compte (par correction du système de chiffrement) si le message chiffré est m_0 ou m_1 . Ainsi on va légèrement contraindre ces appels à l'oracle (comme on avait légèrement modifié la condition de victoire du jeu d'inforgeabilité dans le chapitre 1 pour exclure les messages demandés à l'oracle des attaques réussies (formalisés avec l'ensemble Q)).

Définition 36. Soit Π , un schéma de chiffrement. On dit que le schéma Π est sécurisé contre les attaques à clairs choisis si et seulement si pour tout *PPT* $\mathcal{A}$, la valeur

$$\text{Adv}_{\mathcal{A}, \Pi}^{\text{cca}}(\lambda) := \left| \Pr[\text{PrivK}_{\mathcal{A}, \Pi}^{\text{cca}}(\lambda) = 1] - \frac{1}{2} \right|$$

est négligeable en λ .

Pour le moment, on ne cherchera pas à construire garantir une telle sécurité et on se contentera de la définition. On verra dans le chapitre 3 comment être résilient face à un tel adversaire.

$\text{PrivK}_{\mathcal{A}, \Pi}^{\text{flux}}(\lambda) :$ $s \xleftarrow{\$} \{0, 1\}^\lambda$ $\eta \leftarrow \Pi.\text{Init}(s)$ $b \leftarrow \mathcal{A}^{\text{ONextBit}()}(1^\lambda)$ Retourner $b \stackrel{?}{=} \text{NextBit}(\eta)$	$\text{ONextBit}() :$ $(\eta', b) \leftarrow \Pi.\text{NextBit}(\eta)$ $\eta := \eta'$ Retourner b
---	---

FIGURE 2.8 – Jeu de sécurité du chiffrement de flux

2.2 Systèmes de chiffrement évolués

Dans cette partie, on ne va pas chercher à gagner en sécurité, mais en efficacité. On remarque en effet que le schéma Π_2 produit des chiffrés qui font le double de la taille du message en clair. De plus, il nécessite durant le processus de chiffrement de produire autant d'aléa que de quantité d'information réellement échangé. Même si cet aléa n'a pas à être secret (il fait partie du chiffré qui est publiquement accessible), il est tout de même fort coûteux à générer⁷.

2.2.1 Chiffrement de flux

Si on se concentre sur ce problème d'aléa, les GPA et les FPA ont la particularité, si on les utilise de manière basique, de créer à peine plus d'aléa que ce qu'ils ont besoin de prendre en entrée.

On va donc chercher à créer une quantité beaucoup plus importante d'aléa avec la même base (qu'on appellera la graine). Puis on xorrera cette chaîne d'aléa avec le message pour le chiffrer en reprenant une fois de plus la stratégie du masque jetable.

Pour modéliser cette stratégie, nous allons créer une nouvelle primitive fondamentale : le chiffrement de flux.

Définition 37. *Un chiffrement de flux est défini par deux fonctions déterministes polynomialement bornées :*

- **Init** : Prend en entrée une graine (une chaîne de bits de la taille du paramètre de sécurité), et renvoie un état.
- **NextBit** : Prend en entrée un état, et renvoie un bit et un nouvel état.

Définition 38. *On dit qu'un chiffrement de flux est sécurisé si et seulement si pour tout PPT $\mathcal{A}$:*

$$\left| \Pr[\text{PrivK}_{\mathcal{A}, \Pi}^{\text{flux}}(\lambda) = 1] - \frac{1}{2} \right| \leq \text{neg}(\lambda).$$

Cette primitive fondamentale peut être utilisée de deux manières :

Utilisation synchrone. L'idée de cette utilisation est de considérer l'état du chiffrement du flux comme une clef secrète et de la mettre à jour autant de fois que nécessaire à chaque qu'on chiffre ou qu'on déchiffre une donnée. Le schéma de chiffrement associé est formalisée dans la figure 2.9. On remarque qu'il ne respecte pas le formalisme de la définition 30, puisque la clef secrète ici est dynamique (elle varie à chaque opération de chiffrement ou de déchiffrement).

Exercice 10. 1. Adapter les définitions 27, 30 à ce nouveau contexte.

2. Redéfinir la sécurité contre les attaques à clairs choisis (définition 34) par rapport aux définitions de la question précédente.

3. Prouver la sécurité de $\Pi_{\text{flux}, \text{sync}}$ (figure 2.9) par rapport à la définition précédente.

Utilisation asynchrone. Pour pouvoir utiliser les chiffrements de flux de manière plus standard (c-à-d avec une clef statique), on va devoir étendre notre définition. On regardera ça plus en détails avec l'exercice 20.

⁷. Avec le risque si cet aléa n'est pas choisi assez uniformément d'une faille de sécurité (puisque les prémisses de notre preuve ne seront alors plus valide).

$\Pi_{\text{flux, sync}}.\text{KeyGen}(1^\lambda) :$	$\Pi_{\text{flux, sync}}.\text{Enc}(sk, m) :$	$\Pi_{\text{flux, sync}}.\text{Dec}(sk, c) :$
$s \xleftarrow{\$} \{0, 1\}^\lambda$	$\eta_0 := sk$	$\eta_0 := sk$
Retourner $(\text{Init}(s))$	Pour $i \in \llbracket 1; m \rrbracket :$	Pour $i \in \llbracket 1; c \rrbracket :$
	$(\eta_i, b_i) := \text{NextBit}(\eta_{i-1})$	$(\eta_i, b_i) := \text{NextBit}(\eta_{i-1})$
	Mettre à jour $sk := \eta_m$	Mettre à jour $sk := \eta_m$
	Retourner	Retourner
	$(b_1 \oplus m_1, \dots, b_{ m } \oplus m_{ m })$	$(b_1 \oplus c_1, \dots, b_{ c } \oplus c_{ c })$

FIGURE 2.9 – Schéma de chiffrement synchrone basé sur un chiffrement de flux

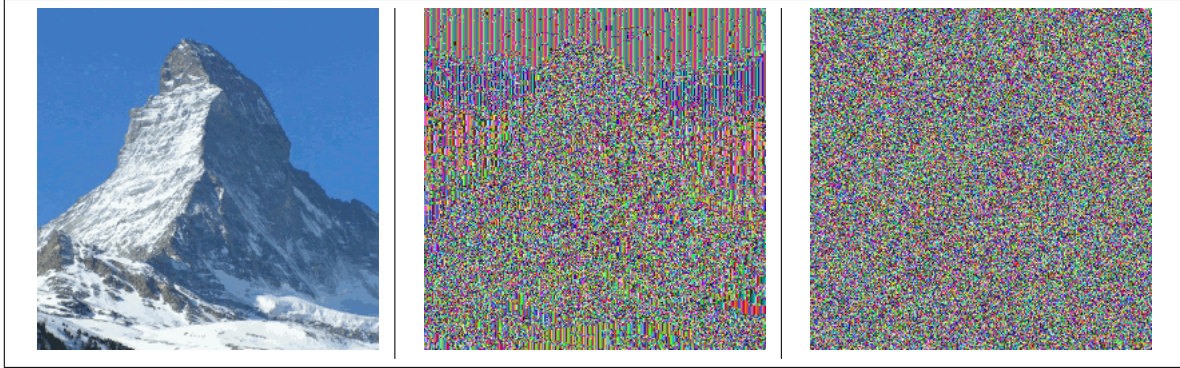


FIGURE 2.10 – La même image en clair, puis chiffrée avec le mode ECB, puis avec le mode CBC (avec DES comme PRP)

2.2.2 Chiffrement par blocs

Dans cette section, on va avoir la même ambition que dans la partie précédente, mais en réutilisant les primitives de la section 2.1.3 : les fonctions pseudo-aléatoires et les permutations pseudo-aléatoires (PPA). Le point commun entre toutes les méthodes de chiffrement par blocs, et de découper les messages en blocs de taille constante (ou dépendant du paramètre de sécurité) puis de générer un chiffré qui est le résultat d’une concaténation de blocs de taille analogue. On pourrait définir formellement ce qu’est un schéma de chiffrement par blocs, mais étant donné que ce n’est pas nécessaire et qu’il y a déjà beaucoup de définitions formelles dans ce cours, on se contentera de décrire formellement les méthodes de chiffrement par blocs les plus connus (et pour ceux qui sont en manque de formalisme, on pourra toujours regarder l’exercice 22). On notera quand dans tous les cas suivants, la génération de la clef consiste à tirer uniformément une clef k de taille λ qui correspond à la clef de la FPA (ou de la PPA) vue dans la définition 35.

Remarque : Ici pour simplifier nos modèles on considère une taille de blocs identique à celle des clefs (en l’occurrence égale au paramètre de sécurité). Dans le cas général, ce n’est pas toujours le cas, par exemple les tailles de blocs pour AES sont identiques pour tous les niveaux de sécurité définis. Cette simplification ne change pas grand chose à nos raisonnements et analyses.

Mode ECB (Electronic Code Book). Ce mode est le plus “naturel”. Il consiste tout simplement à appliquer à chaque bloc la PPA F_k (cf la description de l’algorithme de chiffrement sur la figure 2.11 ou sa représentation graphique figure 2.12). Si cette méthode a l’avantage d’être particulièrement simple, et optimisée au bit près en terme de compacité⁸, elle a le lourd inconvénient d’être déterministe, et donc de chiffrer toujours de la même manière le même message. Et donc elle ne pourra pas vérifier la sécurité contre des attaques à clairs choisis (cf exercice 16 pour plus de détails). Pire même pour une unique utilisation, un même bloc déjà chiffré sera reconnaissable, ce qui est particulièrement problématique par exemple dans un contexte de chiffrement d’images comme on peut le constater avec la figure 2.10. Plus formellement, on peut montrer que ce mode de chiffrement par bloc ne vérifie pas la sécurité contre l’indiscrétion.

8. On remarque en effet que la taille du chiffré fait exactement la taille du texte clair.

$\Pi_{\text{ECB}}.\text{Enc}_k(m_1 \dots m_\ell) :$ Pour $i \in \llbracket 1, \dots, \ell \rrbracket :$ $c_i := F_k(m_i)$ Retourner $(c_1 \dots c_\ell)$	$\Pi_{\text{OFB}}.\text{Enc}_k(m_1 \dots m_\ell) :$ $c_0 \xleftarrow{\$} \{0, 1\}^\lambda$ $f_0 := c_0$ Pour $i \in \llbracket 1, \dots, \ell \rrbracket :$ $f_i := F_k(f_{i-1})$ $c_i := m_i \oplus f_i$ Retourner $(c_0 c_1 \dots c_\ell)$	$\Pi_{\text{CBC}}.\text{Enc}_k(m_1 \dots m_\ell) :$ $c_0 \xleftarrow{\$} \{0, 1\}^\lambda$ Pour $i \in \llbracket 1, \dots, \ell \rrbracket :$ $c_i := F_k(c_{i-1} \oplus m_i)$ Retourner $(c_0 c_1 \dots c_\ell)$
---	---	---

FIGURE 2.11 – Algorithmes de chiffrement par blocs des modes ECB, OFB, et CBC

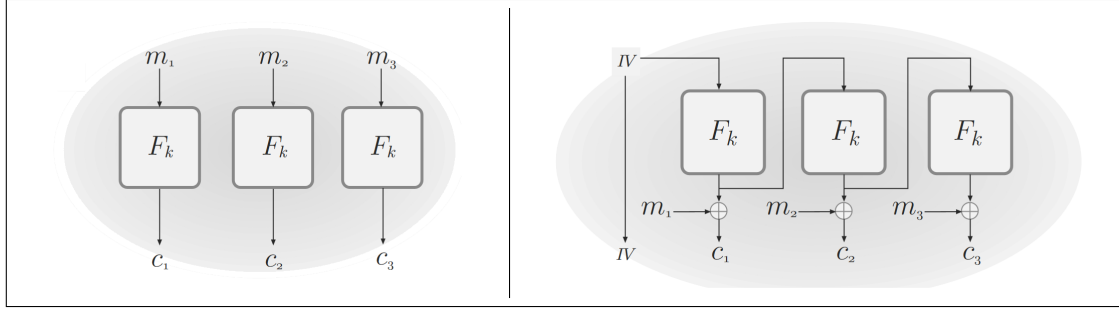


FIGURE 2.12 – Représentation graphique des modes ECB, et OFB

Pour les constructions suivantes, il est acté que l'algorithme de chiffrement ne doit pas être déterministe. L'aléa du chiffrement, qui correspond au r du schéma Π_2 (cf figure 2.1), sera divulgué similairement à la construction Π_2 en constituant le premier bloc du chiffré c_0 . Assez souvent, dans la littérature, ce bloc sera noté IV pour *Initial Vector* (vecteur d'initialisation dans la langue de Molière). De fait les trois modes présentés ici sont sécurisés contre les attaques à clairs choisis (mais les preuves ne seront pas abordées ici).

Mode OFB (Output FeedBack). Ce mode (cf la description de l'algorithme de chiffrement sur la figure 2.11 ou sa représentation graphique figure 2.12) se rapproche des chiffrement par flux : Il consiste à générer une longue suite de blocs à partir du vecteur d'initialisation c_0 en lui appliquant des itérations de la FPA F_k : $(F_k(c_0), F_k^2(c_0), \dots, F_k^\ell(c_0))$. Puis à xorer cette suite de blocs avec notre message et renvoyer le résultat de ce xor précédé du vecteur d'initialisation.

Pour déchiffrer, il suffit de générer la même suite de blocs à partir du vecteur d'initialisation puis de la xorer avec le reste du chiffré pour retrouver le message en clair.

Il a l'avantage d'être résilient aux erreurs de transmission : En effet, si un bloc du chiffré distinct du vecteur d'initialisation est altéré (par du bruit électromagnétique par exemple) l'essentiel du message en clair est obtenu sans erreur après déchiffrement. Dans certains contextes (par exemple pour une vidéo où une erreur d'un pixel ne sera même pas détectée par le spectateur), c'est une propriété très intéressante.

En revanche il ne permet pas de déchiffrer des morceaux arbitraires d'un même message de manière efficace (on doit toujours recalculer les itérés de F_k à partir du premier bloc).

On remarque également qu'on peut précalculer des "proto-chiffrés" sans connaître le message, ce qui dans certains contextes peut avoir un intérêt⁹.

Mode CBC (Cipher Block Chaining). Pour ce mode, on a besoin d'une PPA : On considère le vecteur d'initialisation comme un premier bloc. Puis on xor chaque bloc du message avec le résultat du chiffrement du bloc précédent avant d'en calculer l'image par le PPA F_k (cf figure 2.11). On laisse en exercice au lecteur la rédaction de l'algorithme de déchiffrement.

On remarque que ce mode est strictement moins bien que le précédent en terme de résilience au bruit.

9. On rappelle que le xor est une opération très efficace numériquement (une seule opération assembleur dans la plupart des architectures des ordinateurs modernes), et qu'en revanche l'évaluation de F_k peut être en pratique un peu lourde.

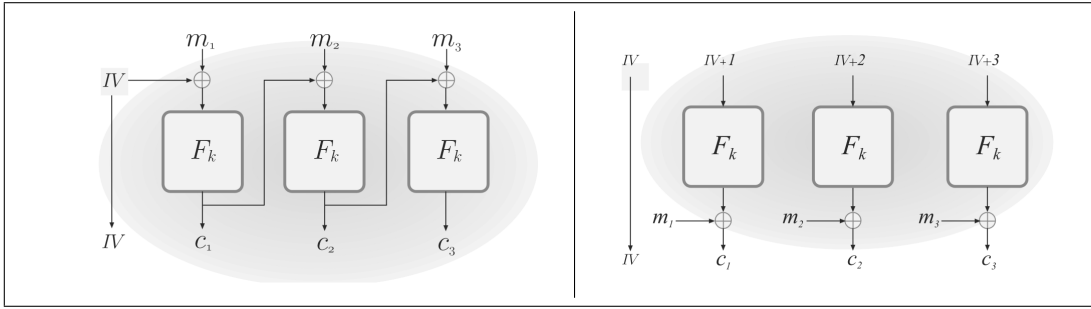


FIGURE 2.13 – Représentation graphique des modes CBC, et CTR

Mode CTR (Compteur). Comme pour le mode OFB, ce mode peut également être vu comme d’inspiration “chiffrement de flux”. Mais pour calculer notre flux pseudo-aléatoire, au lieu de calculer des itérés de F_k on calcule simplement l’image par F_k des ℓ entiers consécutifs à partir du vecteur d’initialisation.

En plus d’être précalculable et résilient au bruit, cette méthode à l’avantage d’être parallélisable : on peut calculer simultanément les images par F_k alors que pour le mode OFB, il fallait les calculer les uns après les autres.

On remarque également qu’on peut faire des déchiffrements partiels de messages de manière efficace.

2.3 Exercices

Exercice 11. On suppose dans cet exercice l’existence d’un système de chiffrement correct Π sur l’ensemble des messages $\mathcal{M} = \{0, 1\}^*$. On pose $\forall N \in \mathbb{N} : \mathcal{M}_N = \cup_{i=0}^N \{0, 1\}^i$.

1. Montrer que $\forall k \in \{\Pi.\text{KeyGen}()\}, \forall m_0, m_1 \in \mathcal{M} : \{\Pi.\text{Enc}_k(m_0)\} \cap \{\Pi.\text{Enc}_k(m_1)\} = \emptyset$.
2. En déduire que pour n assez grand, $p_n := \Pr_{m \leftarrow \mathcal{M}_n} [|\Pi.\text{Enc}_k(m)| \geq \frac{n}{2}] > \frac{3}{4}$.
3. Montrer qu’il existe un $d \in \mathbb{N}[X]$, tel que $\forall \lambda \in \mathbb{N} : \max_{k \in \text{KeyGen}(1^\lambda)} |\{\Pi.\text{Enc}_k(0)\}| \leq d(\lambda)$.
4. Déduire des deux questions précédentes que Π ne peut pas être sécurisé contre les attaques à clairs choisis.
5. Transformer la condition de victoire du jeu de sécurité à clairs choisis pour pouvoir théoriquement couvrir le cas où $\mathcal{M} = \{0, 1\}^*$.

Exercice 12 (Définitions naïves (1)). On constate qu’en pratique, les cryptanalystes essaient de retrouver la clef à partir de couples (messages, cryptogrammes) où les messages sont choisis, et ne considère pas les jeux d’indistingabilité.

1. Définir une définition de sécurité qui colle plus à ces attaques.
2. Montrer que dans le cas d’un schéma correct, la sécurité contre les attaques à clairs choisis implique la sécurité définie ci-dessus.
3. Construire un schéma trivial qui satisfait la sécurité définie dans cette exercice, mais qui n’a aucun intérêt en pratique.

Exercice 13 (Définitions naïves (2)). 1. On peut commencer par considérer la définition suivante qui nous dit qu’en moyenne il est difficile de retrouver intégralement le message chiffré :

Définition 39. On dit qu’un schéma Π est sécurisé naïvement si pour tout adversaire probabiliste polynomialement borné $\mathcal{A}$:

$$\Pr[\text{PrivK}_{\mathcal{A}, \Pi}^{\text{naïf}}(\lambda) = 1] = \text{neg}(\lambda)$$

dans lequel $\text{PrivK}^{\text{naïf}}$ fait référence au jeu défini dans la figure 2.14.

Quelle est la condition implicite sur $|\mathcal{M}|$ pour que la définition soit opérante ?

2. On suppose que $\mathcal{M}$ est une constante indépendante du paramètre de sécurité. Définir alors un adversaire qui rend tout système non sécurisé par rapport à la définition 39.

$\text{PrivK}_{\mathcal{A},\Pi}^{\text{naïf}}(\lambda) :$ $k \leftarrow \Pi.\text{KeyGen}(1^\lambda)$ $m \xleftarrow{\$} \mathcal{M}$ $m' \leftarrow \mathcal{A}(\Pi.\text{Enc}_k(m))$ Retourner $m \stackrel{?}{=} m'$	$\text{PrivK}_{\mathcal{A},\Pi}^{\text{eavd}}(\lambda) :$ $k \leftarrow \Pi.\text{KeyGen}(1^\lambda)$ $(d_0, d_1, \eta) \leftarrow \mathcal{A}_1(1^\lambda)$ $b \xleftarrow{\$} \{0, 1\}$ $m \xleftarrow{d_b} \mathcal{M}$ $b' \leftarrow \mathcal{A}_2(\Pi.\text{Enc}_k(m), \eta)$ Retourner $b \stackrel{?}{=} b'$
---	--

FIGURE 2.14 – Jeu de confidentialité naïf, et jeu d’indistingabilité des distributions

$\text{PrivK}_{\mathcal{A},\Pi}^{\text{LR-cpa}}(\lambda) :$ $k \leftarrow \Pi.\text{KeyGen}(1^\lambda)$ $b \xleftarrow{\$} \{0, 1\}$ $b' \leftarrow \mathcal{A}^{LR_{k,b}(\cdot, \cdot)}(1^\lambda)$ Retourner $b \stackrel{?}{=} b'$	$LR_{k,b}(m_0, m_1) :$ Si $ m_0 \neq m_1 $: Retourner $\perp$ Sinon : Retourner $\Pi.\text{Enc}_k(m_b)$
---	--

FIGURE 2.15 – Jeu d’indistingabilité Gauche ou Droite, et le schéma Π_2

3. On considère alors cette nouvelle définition :

Définition 40. On dit qu’un schéma Π est sécurisé naïvement si pour tout adversaire probabiliste polynomialement borné $\mathcal{A}$:

$$\left| \Pr[\text{PrivK}_{\mathcal{A},\Pi}^{\text{naïf}}(\lambda) = 1] - \frac{1}{|\mathcal{M}|} \right| = \text{neg}(\lambda)$$

dans lequel $\text{PrivK}^{\text{naïf}}$ fait référence au jeu défini dans la figure 2.14.

4. Construire un schéma Π , qui vérifie cette propriété, mais ne vérifie pas la sécurité contre l’indiscrétion.
5. On définit cette nouvelle propriété de sécurité :

Définition 41. On dit qu’un schéma Π est sécurisé naïvement si pour tout adversaire probabiliste polynomialement borné $\mathcal{A}$:

$$\Pr[\text{PrivK}_{\mathcal{A},\Pi}^{\text{eavd}}(\lambda) = 1] = \text{neg}(\lambda)$$

dans lequel $\text{PrivK}^{\text{eavd}}$ fait référence au jeu défini dans la figure 2.14.

Quel est l’implicite sur (d_0, d_1) ? Est-ce un problème en pratique selon vous ?

6. Montrer que cette définition est équivalente à la sécurité contre l’indiscrétion (définition 31).

Exercice 14 (Paradigme Gauche ou Droite).

Définition 42. Soit Π , un schéma de chiffrement. On dit que le schéma Π est sécurisé contre les attaques à clairs choisis si et seulement si pour tout PPT $\mathcal{A}$, la valeur

$$\text{Adv}_{\mathcal{A},\Pi}^{\text{cpa}}(\lambda) := \left| \Pr[\text{PrivK}_{\mathcal{A},\Pi}^{\text{LR-cpa}}(\lambda) = 1] - \frac{1}{2} \right|$$

est négligeable en λ .

Montrer que cette définition est équivalente à la définition 34.

Exercice 15 (Sécurité Sémantique).

Définition 43. Soit $\ell \in \mathbb{N}$, on pose $\mathcal{M} = \{0, 1\}^\ell$. Soit Π , un système de chiffrement sur $\mathcal{M}$. Alors si pour tout $i \in \{1, \dots, \ell\}$, pour tout PPT $\mathcal{A}$:

$$\Pr[\mathcal{A}(1^\lambda, \Pi.\text{Enc}_k(m)) = m^i] \leq \frac{1}{2} + \text{neg}(\lambda),$$

On dit que le schéma est sémantiquement sécurisé.

Montrer que cette définition est équivalente à la sécurité contre l'indiscrétion (définition 31).

Exercice 16 (Chiffrement déterministe). Soit Π , un système de chiffrement correct.

1. Montrer que si $\Pi.\text{KeyGen}$ est déterministe, alors Π ne vérifie pas la sécurité contre l'indiscrétion.
2. On suppose que $\forall \lambda \in \mathbb{N} : \Pi.\text{KeyGen}$ utilise au plus N_λ bits d'aléa. Montrer que si $N_\lambda = O(\log(\lambda))$, alors Π ne vérifie pas la sécurité contre l'indiscrétion.
3. Montrer que si $\Pi.\text{Enc}$ est déterministe, alors Π ne vérifie pas la sécurité contre les attaques à clairs choisis.
4. On suppose que $\forall \lambda \in \mathbb{N}, \forall k \in \Pi.\text{KeyGen}(1^\lambda) : \Pi.\text{Enc}_k$ utilise au plus N_λ bits d'aléa. Montrer que si $N_\lambda = O(\log(\lambda))$, alors Π ne vérifie pas la sécurité contre les attaques à clairs choisis.

Exercice 17 (Générateurs pseudo-aléatoires). Soit G, G' , deux générateurs pseudo-aléatoires respectivement de paramètres d'expansion ℓ, ℓ' . Dire dans les cas suivants si G_i est nécessairement un GPA, avec une preuve dans le cas positif et un contre-exemple dans le cas négatif.

1. $\forall s \in \{0, 1\}^\lambda : G_1(s_1, \dots, s_\lambda) := G(s_1, \dots, s_{\lfloor \frac{\lambda}{2} \rfloor})$.
2. $\forall s \in \{0, 1\}^\lambda : G_2(s) := G(s) || G'(s)$.
3. $\forall s \in \{0, 1\}^\lambda : G_3(s) := G(s_1, \dots, s_{\lfloor \frac{\lambda}{2} \rfloor}) || G(s_{\lceil \frac{\lambda}{2} \rceil}, \dots, s_\lambda)$.
4. $\forall s \in \{0, 1\}^\lambda : G_4(s) := G(0^\lambda || s)$.

Exercice 18 (FPA). Soit F, F' , deux fonctions pseudo-aléatoires. Dire dans les cas suivants si $F^{(i)}$ est nécessairement une FPA, avec une preuve dans le cas positif et un contre-exemple dans le cas négatif.

1. $\forall k, s \in \{0, 1\}^\lambda : F_k^{(1)}(s) := F_k(0 || s) || F_k(0 || s)$.
2. $\forall k, s \in \{0, 1\}^\lambda : F_k^{(2)}(s) := F_k(0 || s) || F_k(1 || s)$.
3. $\forall k, s \in \{0, 1\}^\lambda : F_k^{(3)}(s) := F_k(0 || s) || F_k(s || 1)$.

Exercice 19 (PPA). Montrer qu'une PPA est une FPA.

Exercice 20 (Schéma de chiffrement asynchrone basé sur le chiffrement de flux). Dans cet exercice, on va essayer d'adapter nos définitions de chiffrement de flux afin de pouvoir utiliser des clefs statiques.

Définition 44. Un chiffrement de flux asynchrone est défini par deux fonctions déterministes polynomialement bornées :

- **Init** : Prend en entrée une graine (une chaîne de bits de la taille du paramètre de sécurité), un vecteur d'initialisation et renvoie un état.
- **NextBit** : Prend en entrée un état, et renvoie un bit et un nouvel état.

1. Adapter la définition de sécurité 38 à cette nouvelle définition.
2. L'idée morale est donc de garder toujours la même graine comme clef secrète, et de varier le IV à chaque chiffrement comme on utilise le r pour le schéma Π_2 . Formaliser cela en écrivant un schéma de chiffrement compatible avec la définition 30.
3. Montrer que ce schéma est sécurisé contre les attaques à clairs choisis.

Exercice 21 (Lien entre GPA, chiffrement par flux, et FPA). 1. Construire un GPA de facteur d'expansion arbitraire ℓ , à partir d'un chiffrement de flux.

2. Construire un chiffrement de Flux à partir d'une FPA.

Exercice 22 (Formalisme du chiffrement par blocs). 1. On peut caractériser un chiffrement par blocs deux fonctions F_1, F_2 . Avec F_1 qui prend comme variables f, c, m , et F_2 qui prend comme variable c et m . Expliquer comment.

2. Donner des conditions nécessaires et suffisantes sur F_1, F_2 pour savoir si la fonction est parallélisable, précalculable, résiliente aux erreurs, et partiellement déchiffable.

Exercice 23 (Mode CFB). 1. Écrire l'algorithme de déchiffrement, en précisant si on a besoin que F soit une FPA ou une PPA.

$$\begin{array}{l} \Pi_{\text{CFB}}.\text{Enc}_k(m_1 || \dots || m_\ell) : \\ c_0 \xleftarrow{\$} \{0, 1\}^\lambda \\ \text{Pour } i \in \llbracket 1, \dots, \ell \rrbracket : \\ c_i := F_k(c_{i-1}) \oplus m_i \\ \text{Retourner } (c_0 || c_1 || \dots || c_\ell) \end{array}$$

FIGURE 2.16 – Algorithme de chiffrement par blocs du mode CFB

2. Décrire les intérêts et les inconvénients de ce système (résilience au bruit, déchiffrement partiel, parallélisation du déchiffrement etc).
3. Montrer que si F est une FPA, alors Π_{CFB} est sécurisé contre les attaques à clairs choisis.

Exercice 24 (Attaques à chiffrés choisis). 1. Prouver que Π_2 n'est pas sécurisé contre les attaques à chiffrés choisis.

2. Prouver que les modes de chiffrement par blocs vus dans la partie 2.2.2 ne sont pas sécurisés contre les attaques à chiffrés choisis.

Exercice 25 (Fonctions pseudo-aléatoires faibles).

Définition 45. On dit qu'une fonction est faiblement pseudo-aléatoire, si et seulement si :

$$\left| \Pr_{k \xleftarrow{\$} \{0, 1\}^\lambda} \left[\mathcal{A}^{F_k^\$}(1^\lambda) = 1 \right] - \Pr_{f \xleftarrow{\$} \{0, 1\}^\lambda} \left[\mathcal{A}^{f^\$}(1^\lambda) = 1 \right] \right| \leq \text{neg}(\lambda).$$

Avec pour toute fonction G à valeur dans $\{0, 1\}^\lambda$, $G^\$(), la fonction qui renvoie $(x, G(x))$ avec x tiré uniformément aléatoire dans $\{0, 1\}^\lambda$.$

1. Montrer qu'une FPA est une FPA faible.
2. Construire une fonction F qui est une FPA faible, mais pas une FPA.
3. Dire sans justification pour les constructions $\Pi_2, \Pi_{\text{ECB}}, \Pi_{\text{OFB}}, \Pi_{\text{CBC}}, \Pi_{\text{CFB}}, \Pi_{\text{CTR}}$ quelles propriétés de sécurité on peut atteindre en supposant seulement que F est faiblement pseudo aléatoire.
4. Montrer pour chacun des schémas qu'on ne peut pas atteindre mieux en construisant des attaques pour les propriétés de sécurité plus fortes en considérant que la fonction utilisée est F construite à la question 2.

Exercice 26 (Chiffrement à nuf). Le but des systèmes de chiffrement à nuf est de ne pas recourir à de l'aléa pour chiffrer mais d'utiliser un nuf, c'est à dire une donnée utilisée une seule fois (nuf pour nombre utilisée une fois)¹⁰. On appelle l'ensemble des nuf $\mathcal{N}$.

Définition 46. Un schéma de chiffrement à nuf est un triplet d'algorithmes $DPT : (\text{KeyGen}, \text{Enc}, \text{Dec})$.

$$\begin{array}{l} \text{KeyGen} : \left\{ \{0, 1\}^\lambda \right\}_{\lambda \in \mathbb{N}} \rightarrow \mathcal{K} \\ \text{Enc} : \mathcal{K} \times \mathcal{N} \times \mathcal{M} \rightarrow \mathcal{C} \\ \text{Dec} : \mathcal{K} \times \mathcal{N} \times \mathcal{C} \rightarrow \mathcal{M}. \end{array}$$

1. Adapter la définition de sécurité 34 pour les chiffrements à nuf.
2. Adapter la définition de sécurité 36 pour les chiffrements à nuf.
3. Construire un schéma de chiffrement à nuf cpa-sécurisé et faire la preuve de sécurité.

10. en anglais, on parle de *nonce* qui vient de *number used once*, nombre utilisé une seule fois en anglais.

Chapitre 3

Authentification de messages

Dans ce chapitre, l'objectif considéré ne va plus être la confidentialité, mais l'authenticité¹ d'une information. On a vu dans le chapitre 1 des modèles de sécurité qui ont cette fonction, et on va donc grandement s'inspirer des modèles de sécurité développés dans ce chapitre.

3.1 CAM : premières définitions et premières constructions

Le contexte est le suivant : des utilisateurs appartenant à une entité commune cherchent à s'assurer de cette appartenance à cette entité lorsqu'ils communiquent. Évidemment, on va supposer que les membres de l'entité partagent un secret commun comme pour les schémas de chiffrement vus dans le chapitre 2. Cette hypothèse qui distingue le contexte des signatures vues dans le chapitre 1 implique qu'on catégorise ce schéma dans la famille de la cryptographie symétrique.

Définition 47 (Code d'Authentification de Messages). *Un code d'authentification de messages est un duo de $PPT : (\text{KeyGen}, \text{Mac})$, avec Mac déterministe :*

$$\begin{aligned}\text{KeyGen} &: \{1^\lambda\}_{\lambda \in \mathbb{N}} \rightarrow \mathcal{K} \\ \text{Mac} &: \mathcal{K} \times \mathcal{M} \rightarrow \mathcal{T}.\end{aligned}$$

L'utilisation est la suivante, un utilisateur qui veut authentifier un message m calcule le code associé $t := \text{Mac}(k, m)$ en utilisant la clef secrète k commune à tous les membres de l'entité concernée.

Un utilisateur qui veut vérifier qu'un message m est bien authentique par rapport au code t , en calculant de son côté $t' := \text{Mac}(k, m)$, puis en vérifiant si $t \stackrel{?}{=} t'$.

Remarque Comme on le verra dans l'exercice ??, on peut généraliser la définition en y ajoutant un algorithme de vérification, et en autorisant Mac à être probabiliste (de manière analogue au schéma de signature vu dans le chapitre 1).

Remarque On remarque que vu l'utilisation qu'on a en tête, à partir du moment où Mac est un algorithme déterministe, toute paire d'algorithmes convient, et on a donc pas besoin de définir de propriétés de correction pour ce type de schéma.

On peut s'intéresser directement à la propriété de sécurité des CAM en reprenant ce qui a été fait avec les signatures dans le chapitre 1.

Définition 48 (Inforgeabilité des CAM). *Un CAM est dit inforgeable si et seulement si :*

$$\Pr [\text{MacForge}_{\mathcal{A}, \Pi}(\lambda) = 1] \leq \text{neg}(\lambda)$$

avec le jeu MacForge défini dans la figure 3.1.

On va pouvoir construire assez rapidement un schéma qui vérifie cette propriété de sécurité en réutilisant les FPA vues dans la définition 35. En effet, si il est dur de prédire $F_k(m)$, il est a fortiori encore plus difficile de calculer $F_k(m)$:

1. On peut voir l'utilisation de l'expression "intégrité", mais étant donné l'ambiguïté de cette notion (qui peut aussi faire référence à la résilience au bruit "naturel"), on parlera plus d'authenticité ici.

$\text{MacForge}_{\mathcal{A}, \Pi}(\lambda) :$ $k \leftarrow \Pi.\text{KeyGen}(1^\lambda)$ $Q \leftarrow \emptyset$ $(m^*, t^*) \leftarrow \mathcal{A}^{OMac_k}(1^\lambda)$ Retourner $\text{Mac}_k(m^*) \stackrel{?}{=} t^* \wedge m^* \notin Q$	$OMac_k(m) :$ $t \leftarrow \Pi.\text{Mac}_k(m)$ $Q := \{m\} \cup Q$ Retourner t	$\Pi_3 :$ Soit F_k une fonction pseudo-aléatoire : $\text{KeyGen}(1^\lambda) :$ $k \xleftarrow{\$} \{0, 1\}^\lambda$ Retourner k $\text{Mac} : t \xleftarrow{\$} F_k(m)$ Retourner t
--	---	---

FIGURE 3.1 – Jeu d'inforgeabilité CAM, et un premier schéma

$\mathcal{B}_{\mathcal{A}}^{O(\cdot)}(1^\lambda) :$ $Q \leftarrow \emptyset$ $(m^*, t^*) \leftarrow \mathcal{A}^{OMac^{O(\cdot)}(\cdot)}(1^\lambda)$ Si $O(m^*) \stackrel{?}{=} t^* \wedge m^* \notin Q$ Renvoyer 1 Sinon Renvoyer 0	$\mathcal{B}'^{O(\cdot)}(1^\lambda) :$ $Q \leftarrow \emptyset$ $(m^*, t^*) \leftarrow \mathcal{A}^{OMac^{O(\cdot)}(\cdot)}(1^\lambda)$ $t^{**} \xleftarrow{\$} \{0, 1\}^\lambda$ Si $t^{**} \stackrel{?}{=} t^* \wedge m^* \notin Q$ Renvoyer 1 Sinon Renvoyer 0	$OMac^{O(\cdot)}(m) :$ $Q := \{m\} \cup Q$ Retourner $O(m)$
---	---	---

FIGURE 3.2 – Attaquants $\mathcal{B}$ et $\mathcal{B}'$ contre le jeu des FPA

Propriété 4. Π_3 (figure 3.4) est un CAM sécurisé.

On remarque tout d'abord que $\mathcal{B}^{F_k(\cdot)}(1^\lambda)$ avec k tiré uniformément dans $\{0, 1\}^\lambda$ suit la même distribution que $\text{MacForge}_{\mathcal{A}, \Pi}(\lambda)$. Donc

$$\Pr[\text{MacForge}_{\mathcal{A}, \Pi}(\lambda) = 1] = \Pr_{k \xleftarrow{\$} \{0, 1\}^\lambda} [\mathcal{B}_{\mathcal{A}}^{F_k(\cdot)}(1^\lambda) = 1]. \quad (3.1)$$

Appelons l'évènement $E := m^* \in Q$. Il est trivial de constater que

$$\Pr_{f \xleftarrow{\$} (\{0, 1\}^\lambda)^{\{0, 1\}^\lambda}} [\mathcal{B}_{\mathcal{A}}^{f(\cdot)}(1^\lambda) = 1 | E] = 0. \quad (3.2)$$

On remarque que lorsqu'on est dans le cas $\bar{E}$, dans l'exécution de $\mathcal{B}^{f(\cdot)}(1^\lambda)$, on se rend compte que $O(m^*)$ suit une distribution uniforme sur $\{0, 1\}^\lambda$ indépendante de m^* et t^* .

$$\begin{aligned} & \Pr_{f \xleftarrow{\$} (\{0, 1\}^\lambda)^{\{0, 1\}^\lambda}} [\mathcal{B}_{\mathcal{A}}^{f(\cdot)}(1^\lambda) = 1 | \bar{E}] \\ &= \Pr_{f \xleftarrow{\$} (\{0, 1\}^\lambda)^{\{0, 1\}^\lambda}} [\mathcal{B}'^{f(\cdot)}(1^\lambda) = 1 | \bar{E}] \\ &= \frac{1}{2^\lambda}. \end{aligned} \quad (3.3)$$

On suppose d'abord que $\Pr[\text{MacForge}_{\mathcal{A}, \Pi}(\lambda) = 1] > \frac{1}{2^\lambda}$. Comme F est une FPA, on déduit de (3.1), et (3.3) qu'il existe une fonction négligeable $\epsilon(\lambda)$, tel que $\Pr[\text{MacForge}_{\mathcal{A}, \Pi}(\lambda) = 1] - \frac{1}{2^\lambda} \leq \epsilon(\lambda)$.

Donc $\Pr[\text{MacForge}_{\mathcal{A}, \Pi}(\lambda) = 1] \leq \frac{1}{2^\lambda} + \epsilon(\lambda)$. Ce qu'il fallait démontrer. $\square$

Comme pour la section 2.2, on souhaite à présent gagner en efficacité sans faire de concession de sécurité, on va donc utiliser une technique très similaire en s'inspirant énormément du chiffrement CBC pour la construction CBC_{mac} (figure 3.3) :

Propriété 5. CBC_{mac} (figure 3.3) est un CAM fortement sécurisé.

Résultat admis. $\square$

Soit (F_k) une fonction pseudo-aléatoire :
 CBC_{mac} :
 $\text{KeyGen}(1^\lambda) : k \xleftarrow{\$} \{0, 1\}^\lambda$; Retourner k
 $\text{Mac}(m) : (0^\lambda, c_1, \dots, c_n) \leftarrow \Pi_{\text{CBC}, F}.\text{Enc}_k(n, m_1, \dots, m_n)$
 avec $c_0 = 0^\lambda$
 Retourner c_n

FIGURE 3.3 – CBC-MAC

3.2 Chiffrement authentifié

Dans cette section, on va chercher à définir les propriétés d'authentification sur un système de chiffrement. On parle d'inforgeabilité lorsqu'il est difficile de construire une signature valide, on va donc naturellement étendre la notion au système de chiffrement en parlant d'inforgeabilité lorsque qu'il est difficile de construire un cryptogramme valide, où lorsqu'on dit "cryptogramme valide", on entend que le déchiffrement renvoie un fichier clair lisible (et donc pas une erreur).

Définition 49. *Un schéma de chiffrement Π est dit inforgeable (ou non malléable) si et seulement si :*

$$\Pr [\text{EncForge}_{\mathcal{A}, \Pi}(\lambda) = 1] \leq \text{neg}(\lambda),$$

avec EncForge (défini figure 3.4).

A présent on peut combiner cette propriété avec la propriété de sécurité 36 pour dire ce qu'on souhaite :

Définition 50. *Un schéma de chiffrement Π est dit authentifié, si il est inforgeable et CCA-sécurisé.*

Pour atteindre cette propriété de sécurité, il paraît judicieux de se reposer sur ce qu'on déjà réussit à obtenir c'est à dire les propriétés 34 et 48. Imaginons donc qu'on a donc un schéma de chiffrement Π_E , et un CAM Π_M . La manière la plus naïve de combiner ces deux schémas reste la concaténation. C'est à dire le fait de considérer de chiffrer m , en calculant $(\Pi_E.\text{Enc}_{k_E}(m), \Pi_M.\text{Mac}_{k_M}(m))$, avec k_E, k_M les clés de sécurité des deux systèmes utilisés. Évidemment, ce système ne va pas être sécurisé dans le cas général, car le code d'authentification peut théoriquement faire fuiter de l'information (cf exercice ??). Pour s'assurer que le code d'authentification ne fasse fuiter aucune information, on peut soit le chiffrer avec Π_E en même temps que le message (on explorera cette idée avec l'exercice ??), soit authentifier un chiffré. C'est cette approche qu'on va retenir avec la construction Π_{ETM} (cf figure 3.4). Et comme on va le montrer avec la propriété suivante, on atteindra les objectifs de sécurité maximaux définis jusqu'à présent.

Propriété 6. *Si Π_E , est CPA-sécurisé, et Π_M est un schéma de CAM inforgeable. Alors Π_{ETM} (cf figure 3.4) est un chiffrement authentifié.*

3.3 Fonctions de hachage et applications

3.3.1 Premières définitions

Dans ce chapitre, on va se concentrer sur une primitive fondamentale : les fonctions de hachage. Le concept de haché doit se comprendre comme celui d'*empreinte cryptographique*. Comme pour toute primitive, il convient de définir formellement de quoi on parle.

Définition 51. *Une fonction de hachage paramétrée : $(\text{Setup}, \text{Hash})$, avec Hash déterministe, ℓ une fonction croissante $\mathbb{N} \rightarrow \mathbb{N}$:*

$$\begin{aligned} \text{Setup} &: \{1^\lambda\}_{\lambda \in \mathbb{N}} \rightarrow \mathcal{K} \\ \text{Hash} &: \mathcal{K} \times \{0, 1\}^* \rightarrow \{0, 1\}^{\ell(\lambda)}. \end{aligned}$$

Si Hash à pour ensemble de départ $\mathcal{K} \times \{0, 1\}^n$ au lieu de $\mathcal{K} \times \{0, 1\}^$ avec n une constante, on parle alors de fonction de hachage à entrée bornée.*

$\text{EncForge}_{\mathcal{A}, \Pi}(\lambda) :$ $k \leftarrow \Pi.\text{KeyGen}(1^\lambda)$ $Q \leftarrow \emptyset$ $(m, t) \leftarrow \mathcal{A}^{\text{Enc}_k}(1^\lambda)$ Retourner $\text{Dec}_k(m) \neq \perp \wedge m \notin Q$	$\text{Enc}_k(m) :$ $c \leftarrow \Pi.\text{Enc}_k(m)$ $Q := \{m\} \cup Q$ Retourner c	$\Pi_{\text{ETM}} :$ Soit Π_E, Π_M , un schéma de chiffrement, et un CAM. $\text{KeyGen}(1^\lambda) :$ $(k_E, k_M) \xleftarrow{\$} \{0, 1\}^{2\lambda}$ Retourner (k_E, k_M) $\text{Enc}_{k_E, k_M}(m) :$ $c \leftarrow \Pi_E.\text{Enc}_{k_E}(m)$ $t \leftarrow \Pi_M.\text{Mac}_{k_M}(c)$ Retourner (c, t) $\text{Dec}_{k_E, k_M}((c_1, c_2)) :$ Si $\Pi_M.\text{Mac}_{k_M}(c_1) \neq c_2 :$ Retourner $\perp$ Sinon : Retourner $\Pi_E.\text{Dec}_{k_E}(c_1)$
--	---	--

FIGURE 3.4 – Jeu d’inforgeabilité pour chiffrement, et la construction Chiffrement-puis-CAM

$\text{Coll}_{\mathcal{A}, \Pi_H}(\lambda) :$ $par \leftarrow \Pi_H.\text{Setup}(1^\lambda)$ $(m_1, m_2) \leftarrow \mathcal{A}(par)$ Retourner $\Pi_H.\text{Hash}^{par}(m_1) \stackrel{?}{=} \Pi_H.\text{Hash}^{par}(m_2)$

FIGURE 3.5 – Jeu cryptographique de la résistance aux collisions

Le principe d’une empreinte est le suivant : elle doit caractériser son sujet de manière compacte. On peut voir l’aspect “caractérisation” de plusieurs manières. Mais la plus naturelle est de penser à une sorte d’“injectivité calculatoire”, c’est à dire qu’il est difficile de trouver deux éléments qui ont la même empreinte.

Définition 52. Une fonction de hachage paramétrée H est dite résistante aux collisions si et seulement si :

$$\Pr [\text{Coll}_{\mathcal{A}, \Pi_H}(\lambda) = 1] \leq \text{neg}(\lambda),$$

avec Coll le jeu défini en figure 3.5.

On remarque une première chose qui différencie nettement les fonctions de hachages de toutes les primitives vues jusqu’à présent : il n’y a *a priori* aucune information secrète du challenger par rapport à l’attaquant. A ce titre, on peut dire qu’il s’agit d’une primitive *encore plus symétrique* que celles vues auparavant (GPA, FPA) où il y avait une asymétrie entre la connaissance des utilisateurs du système (qui possédaient la clef secrète), et un adversaire ignorant de cette information². Pour le dire autrement si les empreintes digitales peuvent être une information sensible, le procédé pour produire des empreintes digitales à partir d’une paire de mains lui n’a rien de confidentiel. Ce qui est important avec cette notion d’empreinte cryptographique, c’est qu’elle puisse caractériser de manière unique (au sens calculatoire et non mathématique) une donnée. On verra par la suite une partie des applications d’une telle fonctionnalité.

3.3.2 Extension du domaine de définition avec la méthode de Merkle-Damgård

Tout d’abord, il faut partir du constat que bien souvent, on ne sait pas construire directement des fonctions de hachage à entrée non bornées.

On va donc voir avec la construction de Merkle-Damgård en figure 3.5, illustrée en figure 3.6 comment briser cette limitation.

2. Même si ça reste plus symétrique que dans la cryptologie asymétrique où il y a une asymétrie de l’information des utilisateurs du système.

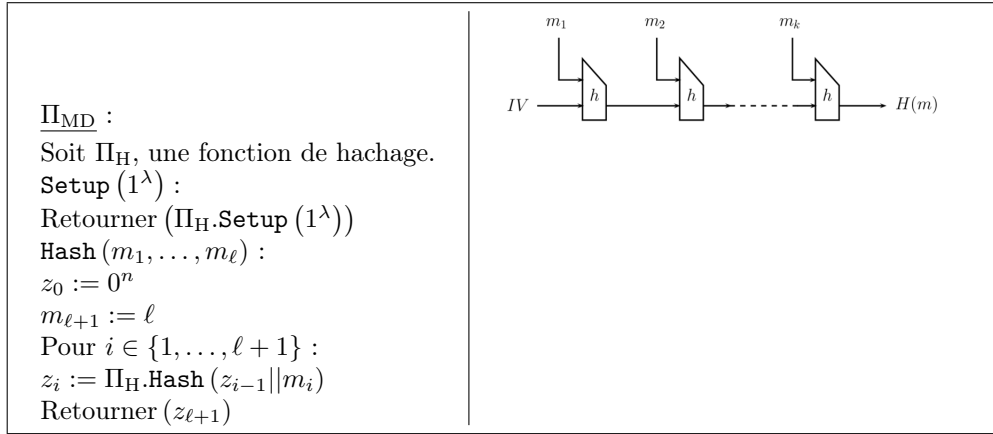


FIGURE 3.6 – Algorithme de Merkle-Damgård, et schéma illustrant ladite construction (sans le rajout de la taille à la fin)

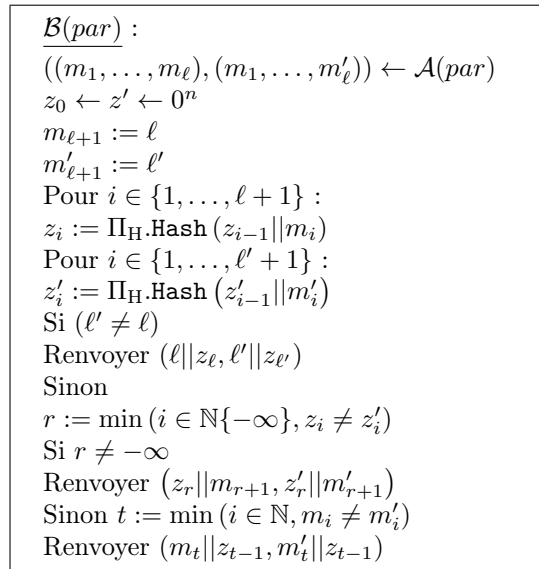


FIGURE 3.7 – Attaquant $\mathcal{B}$ contre la fonction atomique h utilisant un attaquant $\mathcal{A}$ contre Merkle-Damgård

Théorème 5. Soit $(n(\lambda))_{\lambda \in \mathbb{N}} \in \mathbb{N}^{\mathbb{N}}$. Soit Π_H , une fonction de hachage ayant pour ensemble de départ $\{0, 1\}^{2n(\lambda)}$ et ensemble d'arrivée $\{0, 1\}^{n(\lambda)}$. Alors Π_{MD} est une fonction de hachage résistante aux collisions, avec pour ensemble de départ $(\{0, 1\}^{kn(\lambda)})_{k \in \mathbb{N}}$.

On remarque tout d'abord que l'attaquant $\mathcal{B}$ de la figure 3.7 s'exécute en temps polynomial si c'est le cas de $\mathcal{A}$. On peut faire une disjonction de cas pour analyser ce que renvoie $\mathcal{B}$.

1. $\mathcal{A}$ ne renvoie pas une collision (soit car les images des messages sont distinctes), soit car les messages sont identiques. A ce moment là on ne garantit pas le succès de $\mathcal{B}$.
2. $\mathcal{A}$ renvoie une collision et $\ell \neq \ell'$. A ce moment là, on a $z_\ell = z'_{\ell'}$, et $m'_{\ell'+1} \neq m_{\ell+1}$, on en déduit donc que $(\ell || z_\ell) \neq (\ell' || z_{\ell'})$, et $\Pi_H.\text{Hash}(\ell || z_\ell) = \Pi_H.\text{Hash}(\ell' || z_{\ell'})$. Donc $\mathcal{B}$ renvoie bien une collision pour Π_H .
3. $\mathcal{A}$ renvoie une collision et $\ell = \ell'$, et $\exists i \in \llbracket 1; \ell \rrbracket : z_i = z'_i$. A ce moment là, on a $r > -\infty$, comme $z_{\ell+1} = z'_{\ell+1}$ (car $\mathcal{A}$ a renvoyé une collision pour Π_{MD}), et donc par définition de r : $z_r = z'_r$, et $z_{r+1} = z'_{r+1}$ (comme $r \leq \ell$, ces z 's sont bien définis), on a donc : $(z_r || m_{r+1}) \neq (z'_r || m'_{r+1})$, et $\Pi_H.\text{Hash}(z_r || m_{r+1}) = \Pi_H.\text{Hash}(z'_r || m'_{r+1})$. Donc $\mathcal{B}$ renvoie bien une collision pour Π_H .
4. $\mathcal{A}$ renvoie une collision et $\ell = \ell'$, et $\forall i \in \llbracket 1; \ell \rrbracket : z_i = z'_i$. A ce moment là, comme $m \neq m'$, on sait que t est bien défini (c'est un entier naturel), et donc par définition de t , on a $(m_t || z_{t-1}) \neq$

Π_{HTM} :

Soit Π_H, Π_M , une fonction de hachage, et un CAM.

KeyGen (1^λ) :

$par \leftarrow \Pi_H.\text{Setup}(\lambda)$

$k_M \xleftarrow{\$} \Pi_M.\text{KeyGen}(1^\lambda)$

Retourner (par, k_M)

Mac $_{par, k_M}(m)$:

$h \leftarrow \Pi_H.\text{Hash}^{par}(m)$

$t \leftarrow \Pi_M.\text{Mac}_{k_M}(h)$

Retourner (t)

FIGURE 3.8 – Code d’authentification de messages sur $\{0, 1\}^*$ basé sur le paradigme Hash-and-MAC

$\mathcal{B}_1(par)$: $k_M \leftarrow \Pi_M.\text{KeyGen}(1^\lambda)$ $Q := \emptyset$ $(m^*, t^*) \leftarrow \mathcal{A}^{OMac_2(\cdot)}(1^\lambda)$ $h^* \leftarrow \Pi_H.\text{Hash}^{par}(m^*)$ Si $\exists m' (m', h^*) \in Q$ Retourner (m^*, m')	$OMac_1(m)$: $h \leftarrow \Pi_H.\text{Hash}^{par}(m)$ $Q := Q \cup \{(m, h)\}$ $t \leftarrow \Pi_M.\text{Mac}(h)$ Retourner (t)	$\mathcal{B}_2^{OMac}(1^\lambda)$: $par \leftarrow \Pi_H.\text{Setup}(\lambda)$ $(m^*, t^*) \leftarrow \mathcal{A}^{OMac_2(\cdot)}(1^\lambda)$ $h \leftarrow \Pi_H.\text{Hash}^{par}(m)$ Retourner (h, t^*)	$OMac_2()$: $Q := Q \cup \{m\}$ $h \leftarrow \Pi_H.\text{Hash}^{par}(m)$ $t \leftarrow OMac(h)$ Retourner (t)
--	---	--	---

FIGURE 3.9 – Adversaire contre Π_H , puis contre Π_M , utilisant $\mathcal{A}$

$(m'_t || z_{t-1})$ on en déduit donc que $(\ell || z_\ell) \neq (\ell' || z_{\ell'})$, et $\Pi_H.\text{Hash}(\ell || z_\ell) = \Pi_H.\text{Hash}(\ell' || z_{\ell'})$. Donc $\mathcal{B}$ renvoie bien une collision pour Π_H .

On déduit de cette analyse que la probabilité de succès de $\mathcal{B}$ est égale à celle de $\mathcal{A}$. $\square$

3.3.3 Le paradigme Hash-and-MAC

Ainsi le haché d’une donnée la caractérise d’une manière unique (calculatoirement) et compacte. Il va donc de soit qu’authentifier l’empreinte d’une donnée est équivalent à authentifier la donnée elle même. Et comme l’empreinte est de taille constante, on va donc pouvoir utiliser des CAM qui fonctionnent sur des messages de taille constante (comme par exemple la construction Π_3).

Propriété 7. Si Π_H est une fonction de hachage résistante aux collisions à ensemble d’entrées $\{0, 1\}^*$, avec pour ensemble image $\mathcal{M}$, et Π_M un code d’authentification de messages inforgeable sur $\mathcal{M}$. Alors Π_{HTM} (défini figure 3.8) est un CAM fortement sécurisé.

On pose l’évènement $E := \exists m' | (m', h^*) \in Q$.

$$\begin{aligned} \Pr [\text{MacForge}_{\mathcal{A}, \Pi_{\text{HTM}}}(\lambda)] \\ = \Pr [\text{MacForge}_{\mathcal{A}, \Pi_{\text{HTM}}}(\lambda) | E] \Pr[E] + \Pr [\text{MacForge}_{\mathcal{A}, \Pi_{\text{HTM}}}(\lambda) | \bar{E}] \Pr[\bar{E}] \end{aligned} \quad (3.4)$$

On remarque que $\text{MacForge}_{\mathcal{A}, \Pi_{\text{HTM}}}(\lambda)$ implique que $(m^*, \cdot) \notin Q$, alors $\text{MacForge}_{\mathcal{A}, \Pi_{\text{HTM}}}(\lambda) \wedge E \implies m^* \neq m'$. Et dans ce cas, (m^*, m') forme une collision sur Π_H^{par} . On en déduit $\Pr [\text{MacForge}_{\mathcal{A}, \Pi_{\text{HTM}}}(\lambda) | E] \leq \Pr [\text{Coll}_{\mathcal{B}_1, \Pi_H}(\lambda)]$.

On remarque aussi que $\text{MacForge}_{\mathcal{A}, \Pi_{\text{HTM}}}(\lambda)$ implique que $\Pi_M.\text{Mac}(\Pi_H.\text{Hash}(m^*)) = t^*$. Et si on est dans $\neg E$, on en déduit $\text{MacForge}_{\mathcal{B}_2, \Pi_H}(\lambda)$.

De ces deux implications, on peut transformer (3.4) :

$$\begin{aligned} \Pr [\text{MacForge}_{\mathcal{A}, \Pi_{\text{HTM}}}(\lambda)] \\ \leq \Pr [\text{Coll}_{\mathcal{B}_1, \Pi_H}(\lambda)] \Pr[E] + \Pr [\text{MacForge}_{\mathcal{B}_2, \Pi_M}(\lambda)] (1 - \Pr[E]) . \\ \leq \max (\Pr [\text{Coll}_{\mathcal{B}_1, \Pi_H}(\lambda)], \Pr [\text{MacForge}_{\mathcal{B}_2, \Pi_M}(\lambda)]) = \text{neg}(\lambda). \end{aligned}$$

$\square$